

Introduction

The dataset analysed in this project consists of single-cell RNA-sequencing profiles comprising approximately 1,500 cells and over 4,000 genes. It is provided in two complementary files: an .h5ad file containing the full expression matrix and cell-level metadata, and a training CSV file assigning broad hematopoietic lineage labels to roughly two-thirds of the cells. About 500 cells remain unlabeled and form a realistic hidden test set. Preliminary exploratory analysis characterises data quality, sparsity, gene-family composition, and variability, providing essential context for downstream modelling.

To explore the structure of the dataset, a broad range of dimensionality-reduction techniques is evaluated. These include linear Principal Component Analysis (PCA) as well as the non-linear methods t-SNE, UMAP, and autoencoders, each capturing different geometric and biological aspects of the expression landscape. These representations form the basis for extensive clustering experiments aimed at discovering unsupervised groupings in the data.

A diverse collection of clustering algorithms is then applied to each embedding. These include K-means, bisecting (divisive) K-means, and multiple forms of hierarchical clustering with single, complete, and average linkage, combined with several distance metrics such as Euclidean and Manhattan. This systematic comparison allows us to evaluate how different algorithmic assumptions, cluster shape, density, and connectivity, interact with different dimensionality-reduction spaces. Across all tested unsupervised pipelines, the best-performing combination is obtained using an Autoencoder representation, followed by agglomerative clustering with single linkage and Euclidean distance, reaching a silhouette score of 0.603. This configuration surpasses all PCA-, t-SNE-, and UMAP-based pipelines and yields the most compact and coherent cluster separation observed in the analysis.

Following the unsupervised phase, the project turns to supervised learning to classify cells into ten hematopoietic populations. A wide variety of models is tested, including logistic regression, linear and RBF SVMs, multilayer perceptrons, Random Forests, ExtraTrees, XGBoost, and Gaussian Naive Bayes. Each model is evaluated within a rigorous nested cross-validation framework, ensuring no information leakage and providing nearly unbiased performance estimates. In addition, an extensive combinatorial search is performed across preprocessing strategies: different numbers of highly variable genes, filter levels (none, mild, medium, aggressive), pre- vs post-filtering, cell-cycle regression, PCA or autoencoder representations, optional whitening, and the inclusion or omission of curated marker-gene scores.

The final model selected through this procedure is an ExtraTrees ensemble trained on approximately 1,750 HVGs, processed with medium biological filtering and projected onto 20 principal components. This model achieves a macro-F1 score of around 0.44, substantially outperforming naïve or majority baselines and demonstrating that moderately deep tree-based ensembles paired with thoughtful preprocessing can recover biologically meaningful distinctions between closely related hematopoietic lineages

1. Dataset Description

This section provides an exploratory analysis of a single-cell RNA-sequencing dataset that will later be used to develop machine-learning and AI models. The goal of this section is to characterise the basic statistical and biological properties of the dataset. This is a crucial precursor to both unsupervised and supervised modelling: if the data quality is poor or the classes are intrinsically inseparable, no amount of model sophistication will fully compensate.

The data comes in two files: an *.h5ad* file containing the expression matrix for ~1,500 cells and all genes; a *singlecell_train.csv* file with cell identifiers and broad cell-type labels for ~1,000 of those cells. The remaining ~500 cells are intentionally left unlabeled and will act as a hidden test set for supervised models. Biologically, the labels in the training file correspond to coarse hematopoietic lineages:

- *LTHSC_broad* and *STHSC_broad*: long-term and short-term hematopoietic stem cells, expected to be transcriptionally primitive and relatively quiescent.
- *LMPP_broad* and *MPP1/2/3_broad*: lymphoid-primed and multipotent progenitors, representing early differentiation stages.
- *CMP_broad*, *GMP_broad*, *MEP_broad*: common myeloid, granulocyte-monocyte, and megakaryocyte-erythroid progenitors, representing increasingly committed myeloid lineages.
- Other: a small residual category absorbing rare or ambiguous cells.

The *.h5ad* file is read into an AnnData object, which contains: an expression matrix of size n cells $\times$ p genes (1,499 cells and 4,290 genes), per-cell metadata stored in *adata.obs*, and per-gene metadata stored in *adata.var*. The training CSV is loaded separately and used later to attach broad lineage labels to the subset of cells for which supervision is available. The expression matrix in the *.h5ad* file is already log_{1p}-normalised. In the analyses below we work directly with these normalised values and apply additional transformations only where explicitly noted.

1.1 Basic per-cell QC metrics

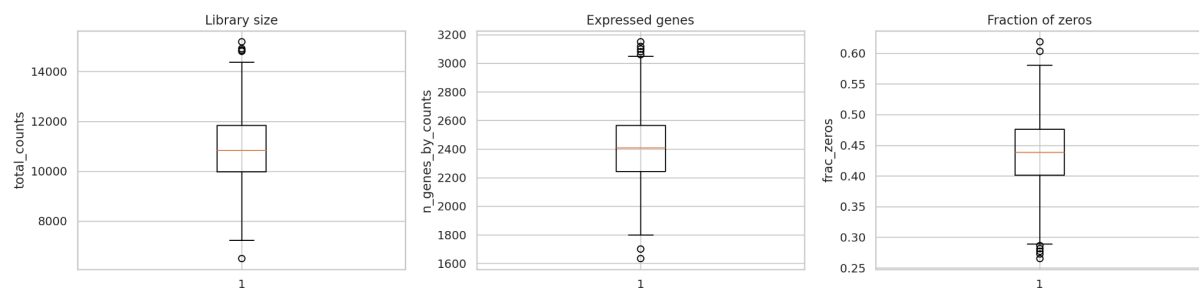
To get a first sense of how well each cell is measured, it is useful to look at three simple quality-control quantities: how many total counts we observe for that cell, how many genes are detected, and how sparse the expression profile is.

Firstly, the library size, so the total number of counts across all genes for a cell is analyzed in the first histogram of Figure 1.1. It reflects how deeply that cell was sequenced, or how much RNA was effectively captured. Library sizes are centred around roughly 11,000 counts, with most cells lying between about 10,000 and 11,800 and a maximum close to

15,000. The number of detected genes, displayed in the second histogram, counts how many genes have expression strictly greater than zero and gives a rough idea of how rich the transcriptome is. Each cell typically shows expression for around 2,200 to 2,600 genes, with a median of about 2,400. The fraction of zero counts (third histogram) measures what proportion of genes are completely silent in that cell and therefore captures how “empty” or “dense” the expression vector is. The fraction of zeros per cell is moderate: the median is approximately 0.44, and the bulk of the cells fall between about 0.28 and 0.58. Overall, the distributions show that almost all cells were sequenced at a reasonable depth. There is no large group of cells with unusually low counts or too many zeros, meaning the dataset does not contain many clearly poor-quality cells.

Figure 1.1

Boxplots of per cell library size, number of detected genes and fraction of zeros



1.2 Per-gene statistics and variability

To understand the basic behaviour of genes in the dataset, we examined how often each gene is expressed, how strong its expression is on average, and how much it varies across cells. The following plots summarise these patterns and show that the dataset contains a mix of genes that are active everywhere, genes that are expressed only in specific cell types, and genes with very different levels of activity. Together, these summaries give a clear picture of the overall structure of gene expression in the dataset.

Figure 1.2.1 shows how often each gene is expressed across all cells. Many genes have very low detection rates: in the first few bins near 0.0, roughly 80-120 genes fall into each bin, meaning they are expressed in only a small fraction of cells. As the detection rate increases, the number of genes per bin stays moderate, usually around 60-100 genes. In contrast, there is a very large peak near 1.0, where almost 380 genes are expressed in almost every cell. Only a smaller number of genes fall in the middle range. Overall, this pattern shows that the dataset contains both rare, cell-type-specific genes and a sizable group of common genes that are active in nearly all cells.

Figure 1.2.1 also shows how often each gene is completely off across the cells. A large group of genes has almost no zeros, meaning they are expressed in nearly every cell (about 380-400 genes). Most other genes have a moderate amount of zeros, with around

80-150 genes in each bin across the middle range. At the other extreme, there is also a noticeable number of genes that are off in almost all cells (about 90-130 genes near 1.0). This tells us that the dataset contains both genes that are active everywhere and genes that are only active in specific cell types.

Furthermore, the distribution of average gene expression across all cells, plotted on a log10 scale. Most genes have fairly low average expression: the left side of the plot shows only a small number of genes with very weak expression (fewer than about 20 genes per bin below -1.5). As expression increases, the number of genes rises steadily, and the largest group appears between about 0.2 and 0.7 on the log scale, where each bin contains roughly 250-360 genes. Only a small number of genes reach the highest expression levels near 1.0. Overall, this means the dataset contains many lowly expressed genes and a smaller group of highly expressed ones, which is typical for single-cell RNA-sequencing data.

Figure 1.2.2 shows the relationship between the average expression of each gene and how much that expression varies across cells, both on a log scale. As expected, genes with higher mean expression also tend to have higher variance, producing an upward trend. Most genes follow this pattern closely, while a few genes deviate from it, appearing either more or less variable than typical for their expression level. This plot helps highlight genes whose variability is unusually high relative to their average expression.

Figure 1.2.1

Per gene detection rate, sparsity and mean expression.

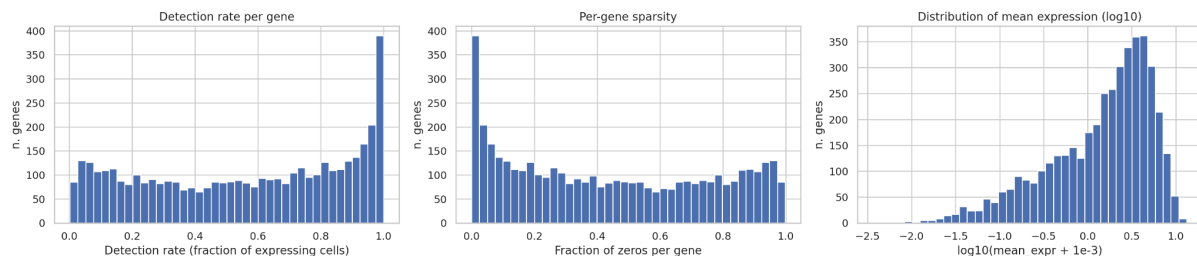
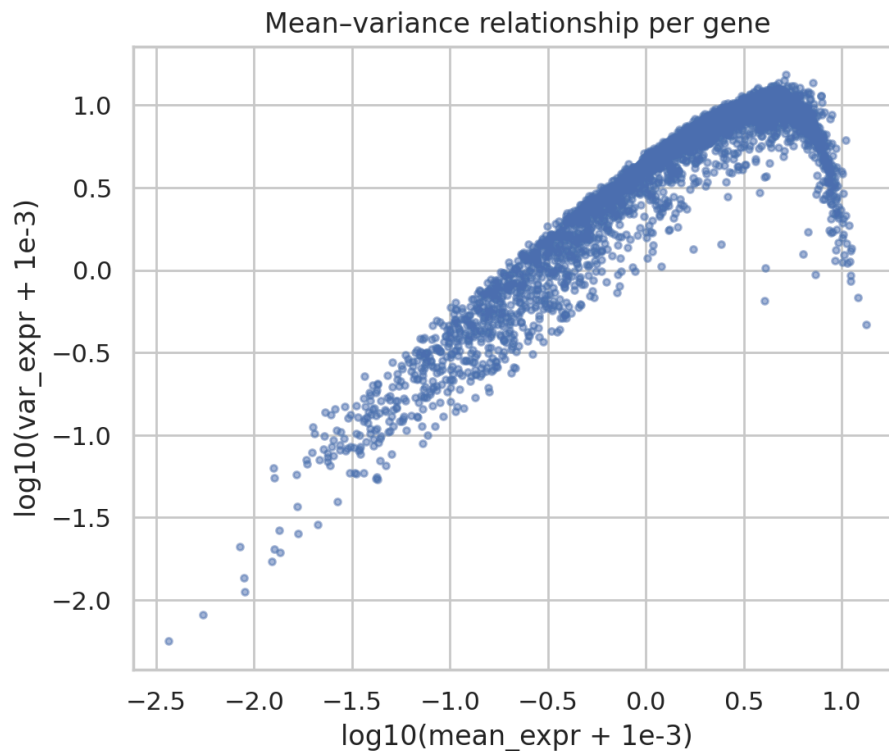


Figure 1.2.2

Mean variance relationship per gene.



1.3 Biological per-cell QC by gene family

To understand how different types of genes contribute to each cell, we grouped the genes into four families: mitochondrial, ribosomal, hemoglobin-related, and immunoglobulin/T-cell receptor genes. We then looked at what percentage of each cell's total RNA comes from each family, displayed in Figure 1.3. This helps reveal which biological signals are strong in the dataset and whether any cell groups stand out because they express unusually high levels of one gene family.

Mitochondrial genes. In the current dataset, the prefix-based rules used to identify mitochondrial genes do not match any of the features, so the corresponding mitochondrial fractions are identically zero.

Ribosomal genes, in contrast, are consistently prominent. Each cell typically expresses about five of the eight ribosomal genes identified, and transcripts from this family account for roughly 10-40% of the total counts, with a median around 25%. This is a substantial but fairly homogeneous contribution across the dataset, in line with the expected housekeeping role of ribosomal components.

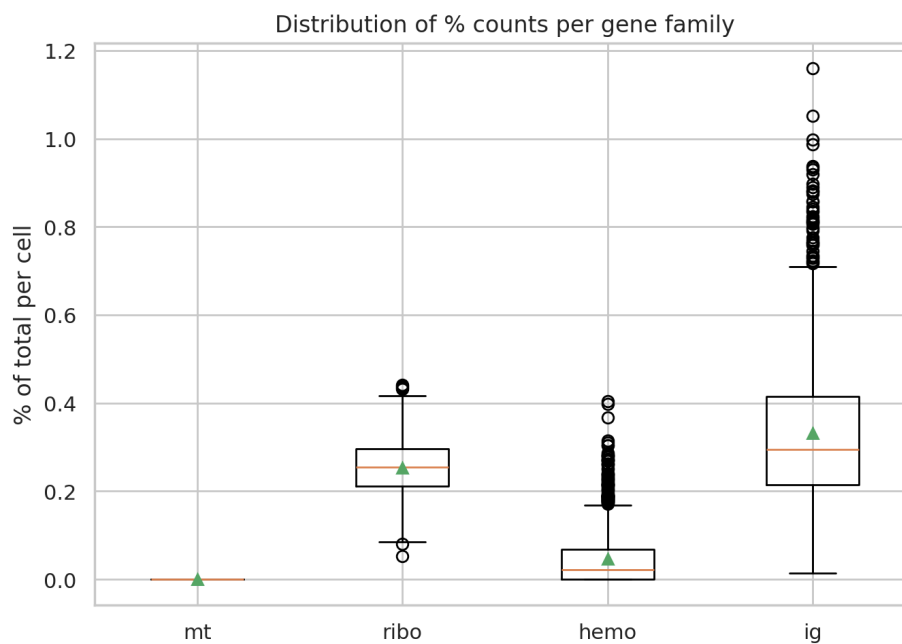
Hemoglobin genes show a much more heterogeneous pattern. Many cells have almost no signal from this family, while a distinct subset reaches up to around 40% of total counts. This mix of cells with almost no hemoglobin signal and others with higher levels suggests that there is a small group of erythroid-like cells in the dataset. These cells might form their own cluster later in the analysis, or they could represent leftover red blood cells, depending on the biological context.

Immunoglobulin and *T-cell receptor genes* are broadly expressed and often dominant. The corresponding transcripts typically contribute between 20% and 40% of the total counts, with a long right tail extending beyond 60%. This pattern is consistent with a strong lymphoid component, rich in B and T cells and antibody or receptor-related activity.

These will serve as keys to build our curated gene filters in Section 1.5.

Figure 1.3

Boxplots of percentage counts per gene family



1.4 Gene-Level Summary: Highly Expressed and Highly Variable Genes

Using the per-gene statistics, we build a compact summary of all features and extract genes with the highest average expression, and genes that show the strongest relative variability (as measured by the squared coefficient of variation).

As shown in Figure 1.4.1, the highly expressed genes, such as *B2m*, *Slc34a2*, *Eif5a*, *Atp5f1b*, *Gpx1*, *Fth1*, and *Npm1*, are mostly housekeeping, metabolic, or structural. They fit

the expected picture of core cellular maintenance functions that are active across many contexts and therefore contribute a large share of the total counts.

The most variable genes, as shown in Figure 1.4.2, according to the squared coefficient of variation, paint a very different picture. This group is dominated by immune-related and receptor genes such as *Ighg3*, *Ighv8-8*, *Ighv14-2*, and *Trbj1-1*, along with a smaller set of signalling and response genes such as *Csf2*, *Arc*, and *Hrh4*. These features are often highly expressed in specific subsets of cells and nearly absent in others, which explains both their high variability and their biological interpretability.

From a machine-learning perspective, this last group is particularly valuable. These highly variable genes are natural candidates for the feature set used in dimensionality reduction (for example, PCA or UMAP) and in clustering algorithms, and they also serve as convenient markers when interpreting model-derived clusters and validating predictions against known biology.

Figure 1.4.1

Top 20 genes by mean expression

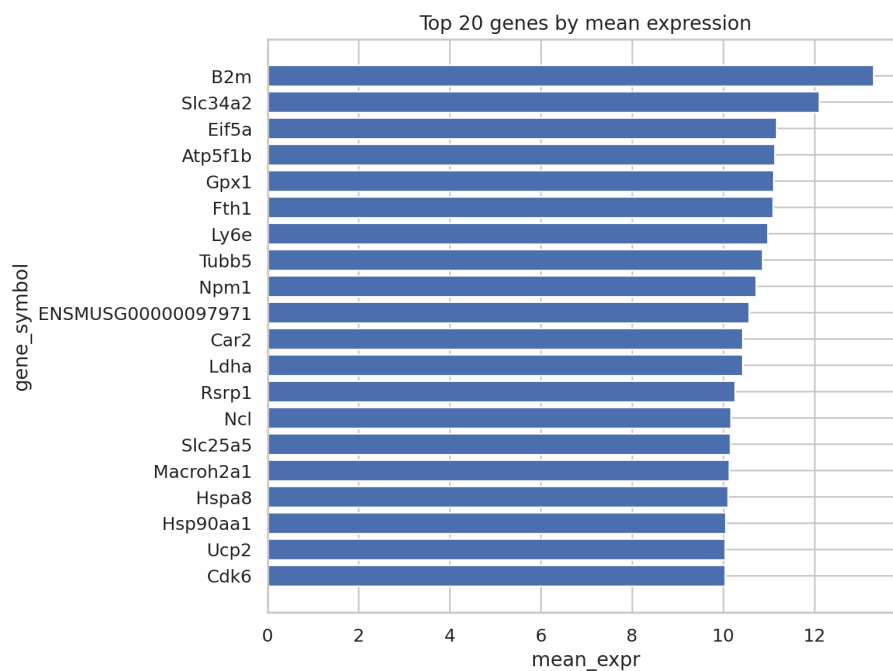
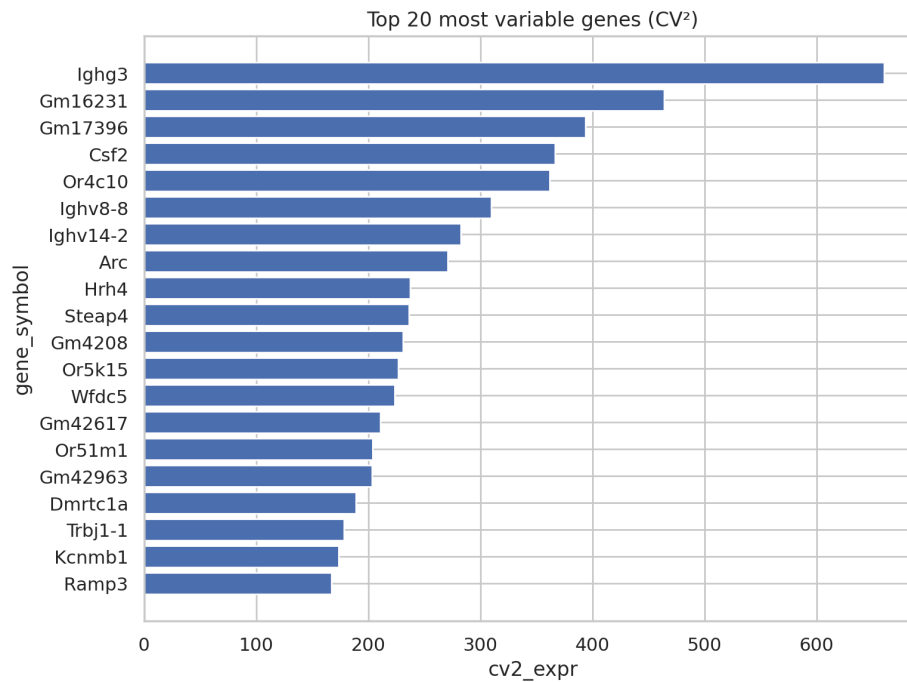


Figure 1.4.2

Top 20 genes by variability



1.5 Biology motivated gene filters

On the account of the above results and the introduction provided in Section 1.3, we purposefully designed a filtering function guided by data as well as some domain knowledge. The filter itself identifies:

- Mitochondrial genes. Genes whose symbols start with *MT-* are treated as mitochondrial.
- Ribosomal genes. Genes whose symbols start with *RPL*, *RPS*, *MRPL* or *MRPS* are flagged as ribosomal.
- Hemoglobin genes. Classical hemoglobin subunits (*HBA1*, *HBA2*, *HBB*, *HBD*, *HBE1*, *HBZ* and related symbols) are grouped together.
- Immunoglobulin genes. Immunoglobulin heavy, light and kappa/lambda chain genes (symbols starting with *IGH*, *IGK*, *IGL* and related forms) are collected into an “Ig” set.
- Cell-cycle genes. Canonical S-phase and G2/M-phase genes are taken from curated lists (the same sets used later for cell-cycle regression in section 3).

- Sex-linked genes. Genes on sex chromosomes, identified from the gene annotation, are grouped together to avoid trivial separation of samples by donor sex.
- Housekeeping-like genes. As a coarse housekeeping proxy we further flag genes that are both very highly expressed and have very low variance across cells. Concretely, we compute per-gene mean and variance; genes above the 95th percentile of the mean and below the 25th percentile of the variance distribution are tagged as “high-mean, low-variance” housekeeping candidates.

These ingredients are combined into four discrete filter levels:

1. none - no genes are removed; all HVGs are eligible.
2. mild - ribosomal and mitochondrial genes are removed.
3. medium - mild filter plus removal of immunoglobulin and hemoglobin genes.
4. aggressive - medium filter plus exclusion of cell-cycle genes, sex-linked genes and the housekeeping-like high-mean/low-variance genes.

1.6 Class distribution and label coverage

The training file contains two key pieces of information: a cell identifier and a broad lineage label. The identifier column can be matched to the cell index in the main AnnData object, while the lineage label distinguishes coarse populations such as LTHSC_broad, CMP_broad, MEP_broad, and related categories. After loading this table, counts per class are computed and visualised as a barplot to give an immediate overview of how many examples are available for each lineage.

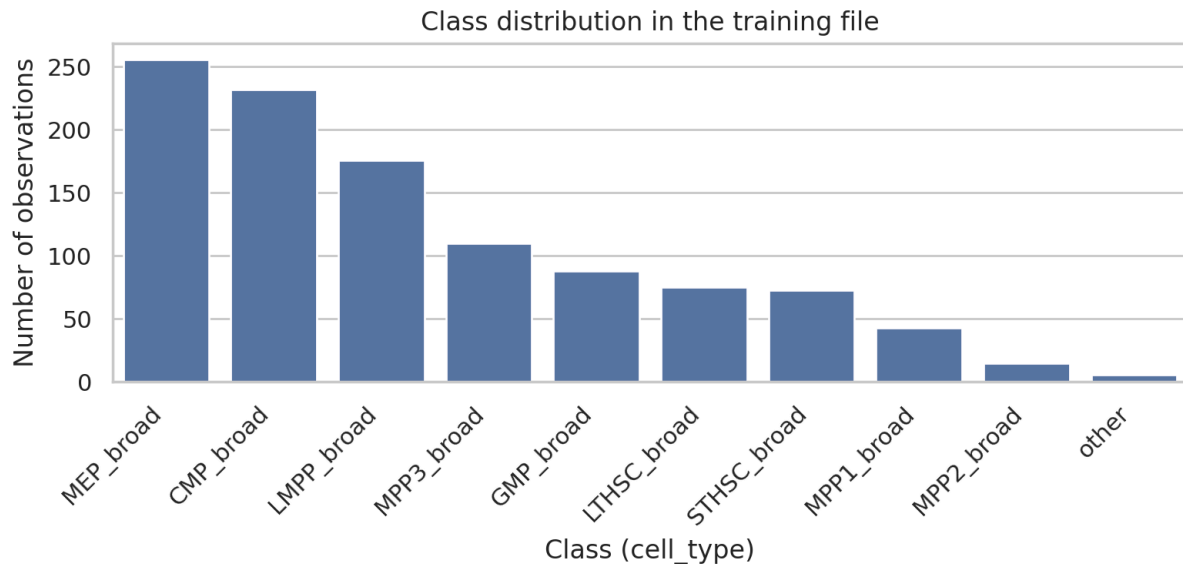
These labels are then mapped back onto the AnnData object by adding a new column in the cell metadata that holds the training label wherever a match is found. This procedure yields labels for roughly two-thirds of the cells; the remaining ~500 cells remain unlabeled and act as a hidden test set. The resulting class distribution is strongly imbalanced. Two categories, MEP_broad (256 cells) and CMP_broad (232), together account for almost half of the labelled cohort. LMPP_broad (176) and MPP3_broad (110) form the next largest groups, followed by intermediate-sized stem-cell populations such as LTHSC_broad (75) and STHSC_broad (73). Other groups are noticeably rarer: MPP1_broad has only 43 cells, MPP2_broad only 15, and the generic “other” category contains just 6 examples.

For supervised learning, this imbalance has several important consequences. A naive focus on overall accuracy will be misleading, because a model that performs well on the abundant classes can still fail almost completely on the rare ones. Metrics that treat classes more symmetrically, such as macro-F1 or balanced accuracy, are more appropriate. To prevent models from effectively ignoring small populations like MPP2_broad or the “other” group, one may need class-weighted loss functions, targeted resampling strategies, or explicit constraints that penalise systematic misclassification of rare classes. Care is also needed when designing cross-validation and test splits to ensure that these small groups are not accidentally excluded from either the training or evaluation sets.

From a semi-supervised or self-supervised perspective, the presence of about 500 unlabeled cells is actually an asset. It creates a realistic setting in which the model must learn from a mixture of labelled and unlabelled data and generalise beyond the limited, imbalanced training labels, using structure in the expression space itself rather than relying solely on the annotated examples.

Figure 1.6

Barplot of the number of labelled cells per cell type.



2. Dimensionality reduction and Clustering

The aim of this part of the project is to systematically explore different combinations of dimensionality-reduction and clustering methods. For each combination, the corresponding clustering results were examined and the configurations that best separated cellular populations and produced clusters consistent with the “true” cell types were selected.

Several families of dimensionality-reduction methods were considered, namely PCA, t-SNE, UMAP and autoencoders, each followed by different clustering algorithms. Every pipeline was evaluated both on the original dataset and on a version pre-processed with the biology-based gene filter, in order to assess the potential benefits of this filtering step. The resulting clusterings were compared in terms of the silhouette score, which provides a quantitative, independent measure of cluster separation.

2.1 Settings

In this section a general overview of the unsupervised models implementation choices are explored.

2.1.1 Clustering

Two different macro-clustering approaches were considered: flat or partitional and hierarchical. The goal is to partition the data into k groups so that the points within each group are as similar as possible, while different groups are dissimilar. The value of k (number of clusters) was based on the number of different cell types.

Flat clustering was performed using the K-means implementation from scikit-learn, with the standard Lloyd algorithm (10 random initialisations and Euclidean distance).

Two forms of hierarchical clustering were considered in this analysis: a bottom-up (agglomerative) approach and a top-down (divisive) approach. This was done to observe whether the data is better represented through progressively merged structures or through successive binary splits. For the hierarchical agglomerative clustering several linkage metrics are tested, more specifically :

- single linkage, where the distance between the clusters is the distance between closest elements in the clusters.

- complete linkage, where distance between clusters is the one between the farthest elements in the clusters, this favours compact and well-separated groups.

- average linkage, with which distance between the clusters is the average of all pairwise distances between elements, which represents a compromise between the two.

In contrast, a divisive strategy was incorporated through bisecting K-means, which recursively splits clusters using K-means with $k=2$. Unlike agglomerative methods, divisive clustering does not rely on linkage definitions: it progressively partitions the dataset by repeatedly selecting and splitting the cluster with the highest within-cluster variance.

2.1.2 Evaluation techniques

To evaluate the quality of the clusters produced by each pipeline, we used the silhouette score. This internal validation index measures, for each cell, how much closer it is to the points in its own cluster than to points in neighbouring clusters. High silhouette values indicate compact, well-separated groups, whereas low or negative values reveal overlapping or poorly defined clusters. Because it depends only on pairwise distances and the cluster assignments, the silhouette score provides an objective, model-independent criterion to compare the many combinations of preprocessing and dimensionality-reduction steps explored in this work.

In all experiments the silhouette score is computed using the same distance notion that is used by the clustering algorithm. For K-means and bisecting K-means this corresponds to the standard Euclidean distance. For hierarchical agglomerative clustering we explore several distance metrics and the silhouette score is always evaluated with the same metric associated

with each specific clustering configuration. This ensures that the numerical evaluation truly reflects the geometry that each algorithm uses to define its clusters.

Alongside this quantitative assessment, the clustering results are also inspected visually by colouring the embedded points both with the predicted cluster labels and with the known biological annotations. Although these true labels are never used during the clustering itself, they provide an external biological reference that helps interpret which pipelines produce clusters that are not only geometrically coherent but also aligned with meaningful cellular populations.

2.2 Principal Component Analysis (PCA)

The first dimensionality-reduction technique that was investigated was Principal Component Analysis (PCA). PCA finds orthogonal directions (principal components) along which the variance of the data is maximised; these correspond to the eigenvectors of the covariance matrix.

For PCA, two different preprocessing strategies were considered:

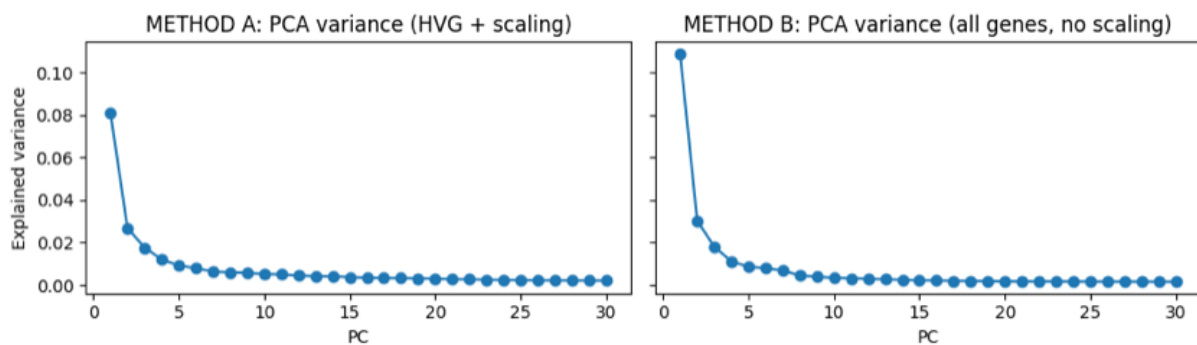
- Method A (full standardisation + HVG selection)
In this setting, the expression matrix was first fully standardised (z-score), so that each gene had mean 0 and variance 1. A highly variable gene (HVG) selection step was then performed. Single-cell RNA-seq data are extremely high-dimensional, and many genes do not meaningfully contribute to distinguishing cell types. To reduce noise and focus on informative features, the top 2,000 highly variable genes were selected using the Seurat v3 HVG selection method. This method was chosen because it efficiently highlights biologically meaningful variation: it computes the mean and variance of each gene, fits the expected mean-variance relationship, and then identifies genes whose variability is significantly higher than expected for their expression level.
- Method B (mean-centering only)
In this alternative preprocessing strategy, only mean-centering was applied and the original gene-wise variances were preserved. This choice was motivated by the idea that z-score standardisation, by dividing each gene by its standard deviation, forces all genes to have unit variance and may down-weight differences in variance that are biologically meaningful. By keeping the original variances, principal components are allowed to reflect the natural variance structure of the data, so that genuinely more variable genes can have a stronger influence on the resulting embedding.

In both cases, PCA was applied using the implementation provided by scikit-learn, which by definition centres the features again before computing the principal components.

To obtain an indication of how many principal components should be retained, the proportion of variance explained by each component, in decreasing order, was computed and plotted. This was done separately for Method A and Method B. In these scree plots, the y-axis represents the percentage of explained variance, while the x-axis corresponds to the principal component index (Figure 2.2). On the basis of these curves, 5, 10, 15 and 20 components were selected for downstream clustering.

Figure 2.2

Scree plots



By combining the two preprocessing strategies (scaled vs. mean-centered), the presence or absence of the biological filter (filtered vs. unfiltered data), and the four choices of the number of principal components, a total of 16 different PCA-based dimensionality-reduction settings were obtained. For each of these 16 representations, the different clustering methods described above were then applied, resulting in 256 distinct pipeline configurations.

Both preprocessing strategies were included because they emphasise different aspects of the data. Method A (full scaling with HVG selection) enforces equal variance for all selected genes, so that each gene contributes on the same scale to the PCA and noise from non-informative genes is reduced. Method B (mean-centering only) preserves the original variances, allowing genes that are naturally more variable to have a stronger impact on the principal components. By testing both approaches, it becomes possible to assess whether clusters are better captured when all genes are equally scaled or when the natural variance structure of the data is retained.

2.2.1 Silhouette score for the 5 best performing combinations

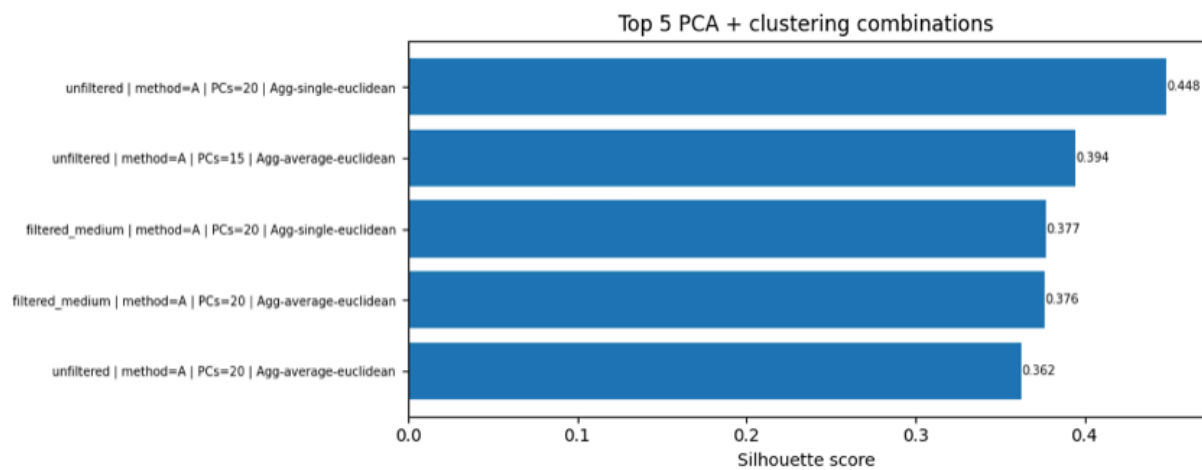
The best five configurations according to the silhouette score all share very similar characteristics. They are obtained with preprocessing method A (highly variable genes followed by z-score scaling), while method B (all genes with centering only) does not appear among the top performers. Moreover, most of the best runs use the unfiltered gene set,

indicating that the medium biological filter does not improve, and may even slightly degrade, the cluster separation in PCA space.

The highest silhouette score (0.448) is achieved with 20 principal components and hierarchical agglomerative clustering using single linkage and Euclidean distance, followed closely by average-linkage agglomerative clustering with 15–20 components. Overall, silhouette values in the range 0.36–0.45 suggest a moderate but meaningful separation: cells tend to form reasonably compact groups in the PCA embedding, although the clusters are not perfectly distinct

Figure 2.2.1

Silhouette score for Top 5 PCA configurations and clustering methods



2.2.2 Unsupervised clustering and comparison to true cell types

Figure 2.2.2 displays the two-dimensional PCA embedding for each of the top five configurations, colouring cells by their predicted cluster assignment. This view makes it possible to visually assess the geometric separation produced by each clustering method. In all top configurations the cells arrange along an elongated, branched manifold rather than forming a set of well-separated spherical clouds. The predicted clustering therefore tends to produce one or two large macro-clusters that follow the main body of the manifold, plus a few smaller groups that isolate peripheral regions of the space. This explains why only a limited number of colours are visibly dominant in the predicted-label plots, even when the number of requested clusters is relatively high. Taken together, this figure provides a purely geometric perspective on cluster quality, independent of the biological ground truth.

Figure 2.2.3 shows the same PCA embedding as Figure 2.2.2, but cells are coloured by their true biological labels (`train_label`). This visualisation allows the degree of correspondence between predicted clusters and actual cell types to be assessed. It becomes

apparent that the main body of the manifold is occupied by a mixture of several populations rather than by a single homogeneous group. In the best-performing clustering configurations, however, almost all of these cells are assigned to one dominant predicted cluster, while only the distal branch of the embedding is split off as a separate group.

Figure 2.2.2

PCA clusters for the top 5 configurations, colored by predicted cluster labels



PCA embeddings for the top 5 configurations, colored by true biological cell types



2.3.1 HVG and Pca

16

PCA is applied to the highly variable gene (HVG) expression matrix to capture the directions of maximum variance in the dataset. Although the HVG selection step reduces the dataset to 2,000 genes, this is still a high-dimensional and noisy space, and applying t-SNE directly would produce an unstable and distorted embedding due to irrelevant variance and technical noise. PCA overcomes this by identifying the major axes of variation, those directions that explain the most meaningful biological differences, and projecting the data onto them. In practice, PCA compresses the dataset into a smaller set of informative components while removing noise. Based on the elbow method described earlier, the first 10 principal components are retained.

Importantly, PCA is not performed once globally. Instead, it is recomputed separately for each of the four dataset conditions generated in the analysis:

1. data without the biological filter and without scaling,
2. data without the biological filter and with scaling,
3. data with the biological filter and without scaling,
4. data with the biological filter and with scaling.

For each condition, the HVG matrix undergoes its own PCA decomposition, ensuring that the dimensionality reduction reflects the specific preprocessing applied. The resulting PCA components are then used as the input for t-SNE to ensure a more stable, efficient, and biologically meaningful visualization

2.3.2 t-SNE as a non linear dimensionality reduction method

After PCA, t-SNE is applied because PCA is a linear dimensionality-reduction method and therefore cannot capture non-linear relationships in the data. Many patterns in single-cell RNA-seq, such as developmental trajectories, branching structures, or subtle cell-type separations, are inherently non-linear and cannot be represented by PCA alone. In contrast, t-SNE is specifically designed to uncover non-linear structure by preserving the local neighborhood relationships of the data: cells that are close in the original high-dimensional gene-expression space remain close to each other in the 2-dimensional embedding. However, t-SNE does not preserve global distances; points that appear far apart in the embedding may not be equally distant in the original gene-expression space. Despite this limitation, representing the data in a 2-D coordinate system (x and y), where each point corresponds to a single cell, makes the high-dimensional data far more interpretable.

In this visual space, points that cluster together correspond to cells with similar expression patterns, allowing biological structure to become visually apparent. Similar cells appear close to each other because t-SNE performs dimensionality reduction by preserving the local structure of the data. In summary, PCA reduces noise and compresses the data, while t-SNE reveals non-linear variation and local structure that PCA cannot capture,

producing a clearer and more biologically meaningful representation of the underlying cell populations.

2.3.3 t-SNE hyperparameters grid search

Because t-SNE is very sensitive to hyperparameters, a grid of different parameter combinations is explored. More precisely, different combinations of perplexity, the learning rate, and the distance metric are tested.

Perplexity represents the number of “effective neighbors” each point considers when t-SNE builds the probability distribution centered around it. A higher value means t-SNE will take more global distances into account, whereas a low value makes it focus mostly on very local relationships. The perplexity values used (15, 30, and 50) cover this range: the smallest value produces smaller and more fragmented clusters, while the largest captures broader, more global variation with larger clusters.

The learning rate controls how large each optimization step is when t-SNE performs gradient descent. Both 200, which produces slower and more stable convergence, and 500, which performs faster but more aggressive convergence, are tested. The objective that t-SNE optimizes during this process is the Kullback-Leibler divergence between the high-dimensional distribution and the 2-D embedding.

The distance metric defines how distances between points are computed before t-SNE converts them into probabilities. Euclidean distance is tested because it is the standard choice and works well when variation is smooth and continuous. Manhattan distance is also tested because it is more robust to outliers and can capture differences better when the data have sharp changes or are not normally distributed, conditions that are common in gene-expression data. This systematic grid search is necessary to understand how t-SNE’s behavior changes depending on the hyperparameters and to identify combinations that produce stable and biologically meaningful embeddings.

2.3.4 Clustering methods on t-SNE embeddings

As outlined in the section above, 11 clustering methods are performed on each configurations of hyperparameters and on each dataset conditions. Different clustering methods capture different shapes, densities, and hierarchical relationships in the data, and evaluating multiple approaches helps avoid bias toward any single algorithm. In total, 4 dataset conditions, 4 t-SNE parameter settings, and 11 clustering methods are tested, yielding 176 combinations. This large search space makes the framework both powerful and comprehensive. To assess the quality of the resulting clusters, the silhouette score is used, consistent with the evaluation approach described in the previous section.

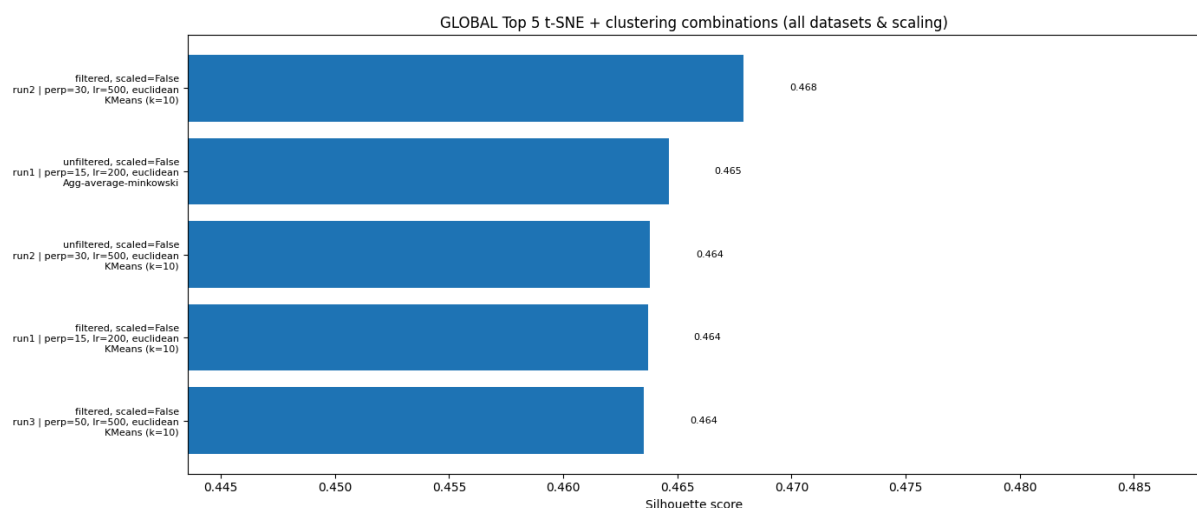
2.3.5 Silhouette score for the 5 best performing combinations

As shown in Figure 2.3.5, the silhouette scores of the top five combinations range from 0.464 to 0.468. The highest clustering quality is obtained with the filtered dataset, without scaling, using a t-SNE configuration with perplexity = 30, learning rate = 500, and Euclidean distance, followed by K-means clustering with $k = 10$. While most of the top-performing configurations use K-means with Euclidean distance, making their silhouette scores directly comparable, one of the top five results is obtained using agglomerative clustering with average linkage. Because silhouette values depend on the distance metric used by the clustering algorithm, scores across different clustering methods should be interpreted with caution. Nevertheless, the overall range of values suggests consistent clustering quality across methods.

Interestingly, all high-performing configurations rely on unscaled data. This is likely because scaling forces all genes to have mean = 0 and variance = 1, which can remove biologically meaningful differences in expression magnitude. When these absolute differences carry true biological signals, scaling may flatten important variation and reduce the separability of cell populations. Both the filtered and unfiltered datasets achieve high silhouette scores, indicating that the removal of technical genes (mitochondrial, ribosomal, and immunoglobulin genes) does not strongly distort the underlying biological structure in this dataset. Finally, K-means generally emerges among the top-performing approaches, likely because t-SNE produces compact, well-separated clusters, a structure that is well suited for algorithms like K-means that partition data around centroids.

Figure 2.3.5

Silhouette score for Top 5 t-SNE configurations and clustering methods



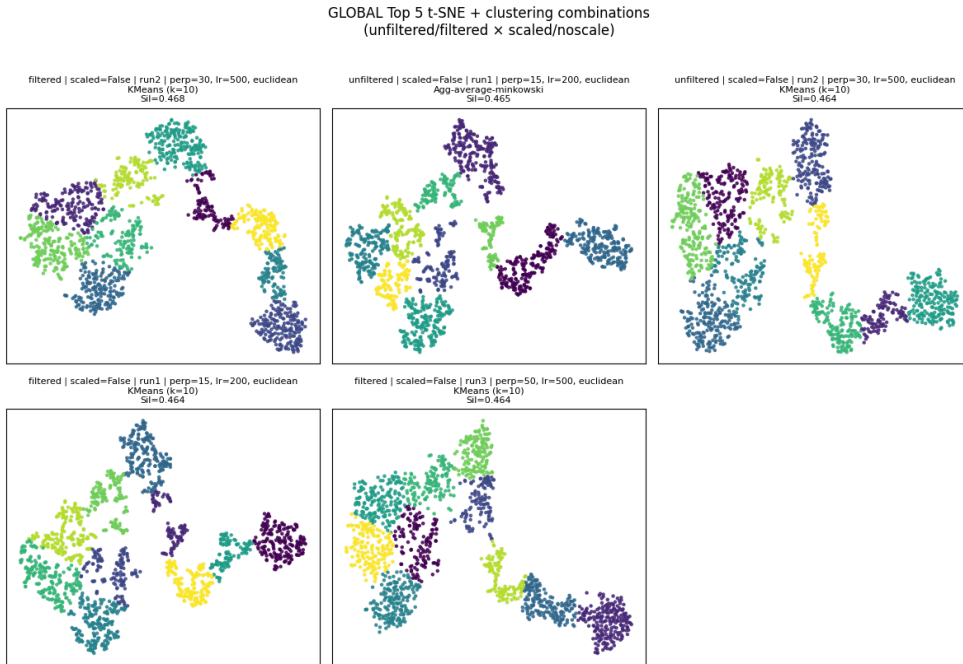
2.3.6 Unsupervised clustering and comparison to true cell types

From Figure 2.3.6, which shows clusters without cell-type labels, we can see that the cells form clearly separated groups in the t-SNE embedding. This indicates that the dataset contains real underlying structure: some cells are more similar to each other than to the rest. However, it is important to note that t-SNE only creates the 2-dimensional layout. The clustering algorithms are what assign labels to these points, meaning they color the t-SNE map but do not determine its shape. Because both t-SNE and the clustering methods are unsupervised, they detect patterns based only on gene-expression similarity, without any knowledge of the true biological identity of the cells.

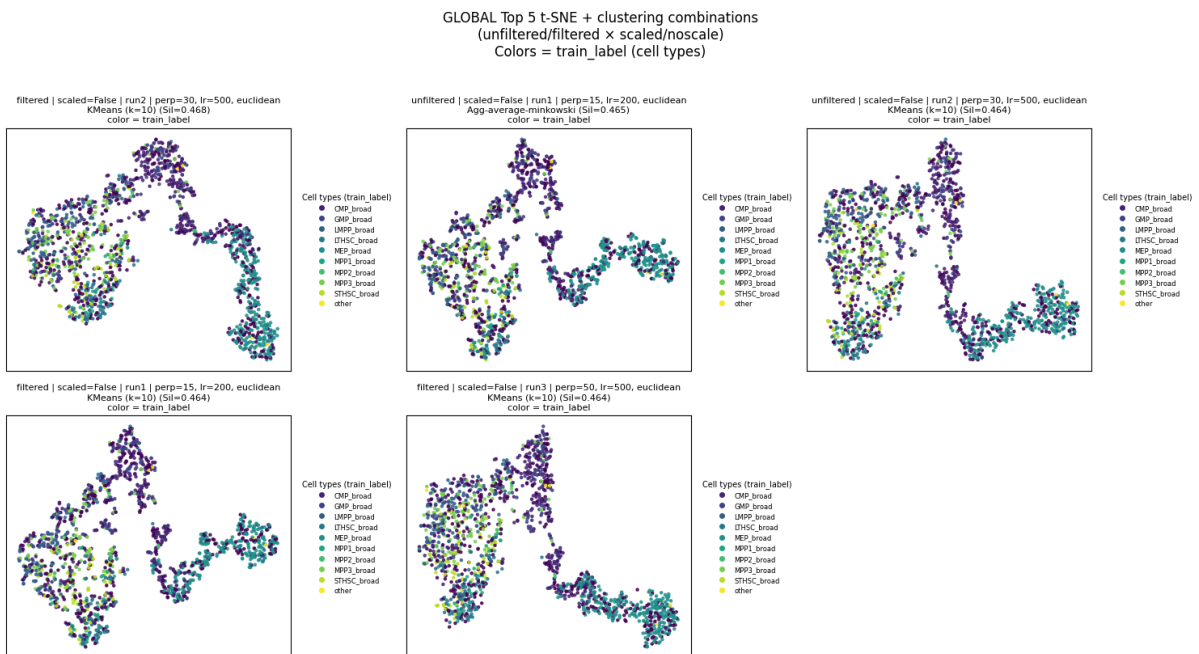
Figure 2.3.7 shows the same t-SNE embeddings, but now the points are colored according to the true biological cell types. Here we see that these annotated cell types do not align with the clusters produced by clustering on the t-SNE embedding. If the discovered clusters matched the real cell types, each color would appear as a single compact region. Instead, the colors are scattered and mixed across the embedding. This means that although t-SNE and the clustering methods are capturing some biological differences, these differences are not the same ones that define the broad annotated cell types. In other words, the unsupervised pipeline is detecting other strong patterns in the data that do not correspond exactly to the provided labels.

Overall, this comparison shows that while the t-SNE and clustering pipeline detects clear structure in the data, this structure does not reflect the broad cell-type labels and therefore must correspond to other underlying biological or technical factors. In the following sections, other techniques of dimensionality reductions are explored, starting with Uniform Manifold Approximation and projection.

t-SNE embeddings for the top 5 configurations, colored by predicted cluster labels



t-SNE embeddings for the top 5 configurations, colored by true biological cell types



2.4 Umap

In this section, UMAP dimensionality reduction is performed using multiple parameter combinations, and each embedding is evaluated with 11 different clustering methods, mirroring the analysis carried out with `pca` and `t-SNE`. This approach allows us to assess how UMAP hyperparameters and clustering algorithms influence the separation of cell populations, and also to compare how different non-linear dimensionality-reduction methods affect clustering quality on the same dataset.

2.4.1 Umap as a non-linear dimensionality reduction method

UMAP is a non-linear dimensionality reduction method that creates a 2-dimensional embedding of the data while preserving meaningful structure from the original high-dimensional space. In this analysis, UMAP is applied after PCA and used on all dataset configurations described previously (filtered vs. non-filtered, scaled vs. non-scaled). PCA is first used as a linear technique to compress the high-dimensional gene-expression matrix into the 10 principal components that capture the largest sources of variance. However, because PCA is purely linear, it cannot model more complex, curved, or non-linear relationships that are characteristic of biological processes. For this reason, UMAP is used as a second step to reveal non-linear structure that PCA alone cannot capture.

UMAP works by constructing a graph of nearest neighbors in the high-dimensional space and then learning a low-dimensional representation that best preserves this graph. It does so by minimizing a cost function that measures how different the high-dimensional and low-dimensional relationships are. This objective function contains two components: an attraction term, which pulls similar points closer together in the 2-D embedding, and a repulsion term, which pushes dissimilar points apart to prevent unrelated cells from collapsing into the same region. This repulsion term is what `t-SNE` lacks in its global formulation, and it allows UMAP to preserve not only local neighborhoods but also aspects of global structure, such as the relative positioning of larger groups of cells. As a result, UMAP tends to produce embeddings that are more stable, more reproducible, and better reflect the broader organization of the dataset. UMAP optimizes its embedding using stochastic gradient descent, iteratively adjusting point positions in low-dimensional space to satisfy these attraction and repulsion forces.

In the resulting UMAP maps, spatial proximity directly reflects similarity in gene-expression profiles: cells with similar transcriptional programs appear close to each other, forming visually interpretable clusters or trajectories. By contrast, while `t-SNE`

embeddings effectively reveal local substructure and compact groupings of similar cells, inter-cluster distances and spatial arrangements lack direct biological interpretation.

2.4.2 Umap hyperparameters grid search

Since UMAP is highly sensitive to its hyperparameters, the analysis is performed using several different parameter configurations. First, different values of the number of neighbors are tested. This parameter defines the size of the local neighborhood used to build the manifold graph and therefore controls how UMAP balances local and global structure. The values 10, 30, and 50 are explored: the smallest value emphasizes very local relationships and produces smaller, tighter clusters, while the largest value incorporates broader global structure and results in smoother, more extended clusters.

The minimum distance parameter controls how close points are allowed to be in the two-dimensional embedding. Lower values generate tighter, more compact clusters, whereas higher values lead to more spread-out representations. The values 0.05, 0.10, and 0.30 used in the analysis therefore cover a range from highly compact to more diffuse layouts.

Finally, the distance metric determines how similarity between cells is computed in the PCA space before UMAP constructs the low-dimensional embedding. Two metrics are tested: Euclidean distance, which is the standard choice for continuous variation, and cosine distance, which focuses on the direction rather than the magnitude of gene expression changes. Testing both allows the analysis to capture either absolute expression differences or relative co-expression patterns.

2.4.3 Silhouette score and selection of the top 5 combinations

Figure 2.4.3 shows the five best-performing combinations of UMAP parameters and clustering methods, ranked by their silhouette score. The highest score (0.558) is obtained with the unfiltered, scaled dataset using UMAP with 30 neighbors, a minimum distance of 0.05, cosine distance, and K-means clustering with ten clusters. The second-best combination (0.556) uses the same UMAP configuration but applies agglomerative clustering with average linkage and the Minkowski metric instead of K-means. Because these clustering algorithms rely on different distance metrics, their silhouette scores are not directly comparable, and the ranking should therefore be interpreted cautiously.

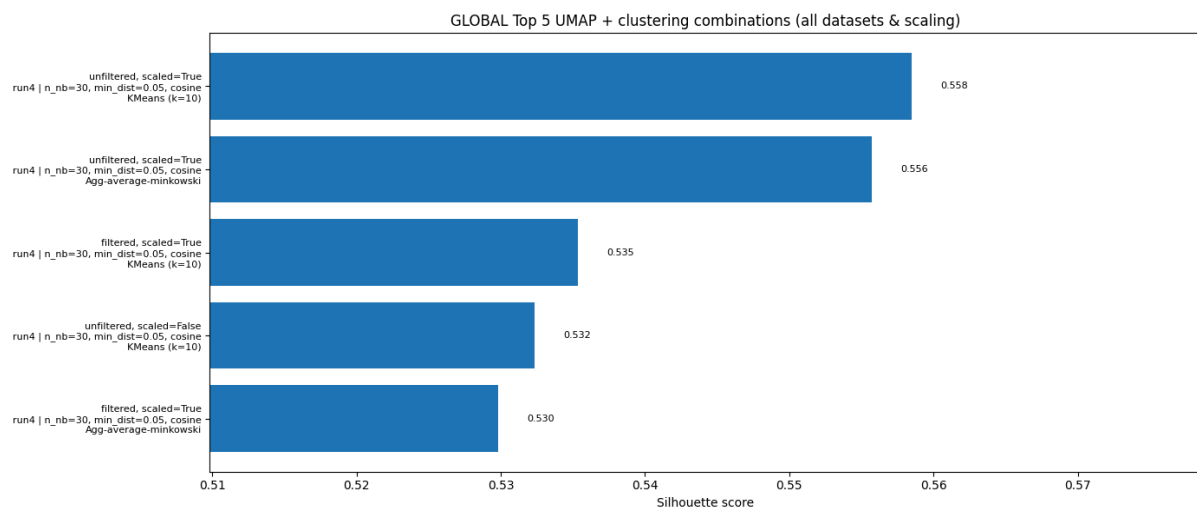
The remaining top configurations also share the same UMAP settings, indicating that this parameter combination reliably produces coherent local structure across preprocessing conditions. Overall, silhouette values between 0.530 and 0.558 suggest moderate but consistent cluster separation, indicating that UMAP captures meaningful biological structure, even though absolute cluster compactness remains modest.

There is a clear interpretation for why this particular set of UMAP parameters consistently achieves the highest silhouette scores. Using 30 neighbors appears to work best because it is small enough to preserve fine-grained local biological structure, while still large enough to capture broader relationships across the manifold rather than breaking the data into overly fragmented regions. A minimum distance of 0.05 forces points to form tight, compact clusters, which is beneficial for clustering methods such as K-means that assume dense, well-defined groups. Finally, cosine distance emphasizes the relative patterns of gene expression rather than absolute expression magnitude, making it more sensitive to biologically meaningful co-expression programs.

By comparing the results from UMAP (2.4.3) with those from t-SNE (Figure 2.4.5), it becomes clear that UMAP achieves higher silhouette scores and produces more coherent cluster boundaries. The top five UMAP configurations reach silhouette scores between 0.530 and 0.558, whereas the best t-SNE configurations only range from 0.464 to 0.468. This indicates that UMAP creates a low-dimensional representation where clusters are more compact and better separated than those produced by t-SNE. As a consequence, clustering algorithms, particularly K-means, perform better on UMAP embeddings. This difference arises because t-SNE preserves only very local neighborhoods and often distorts the global arrangement of points, while UMAP preserves both local and intermediate-scale structure, resulting in more stable and biologically interpretable embeddings.

Figure 2.4.3

Silhouette score for Top 5 UMAP configurations and clustering methods



2.4.4 Unsupervised clustering vs true cell types

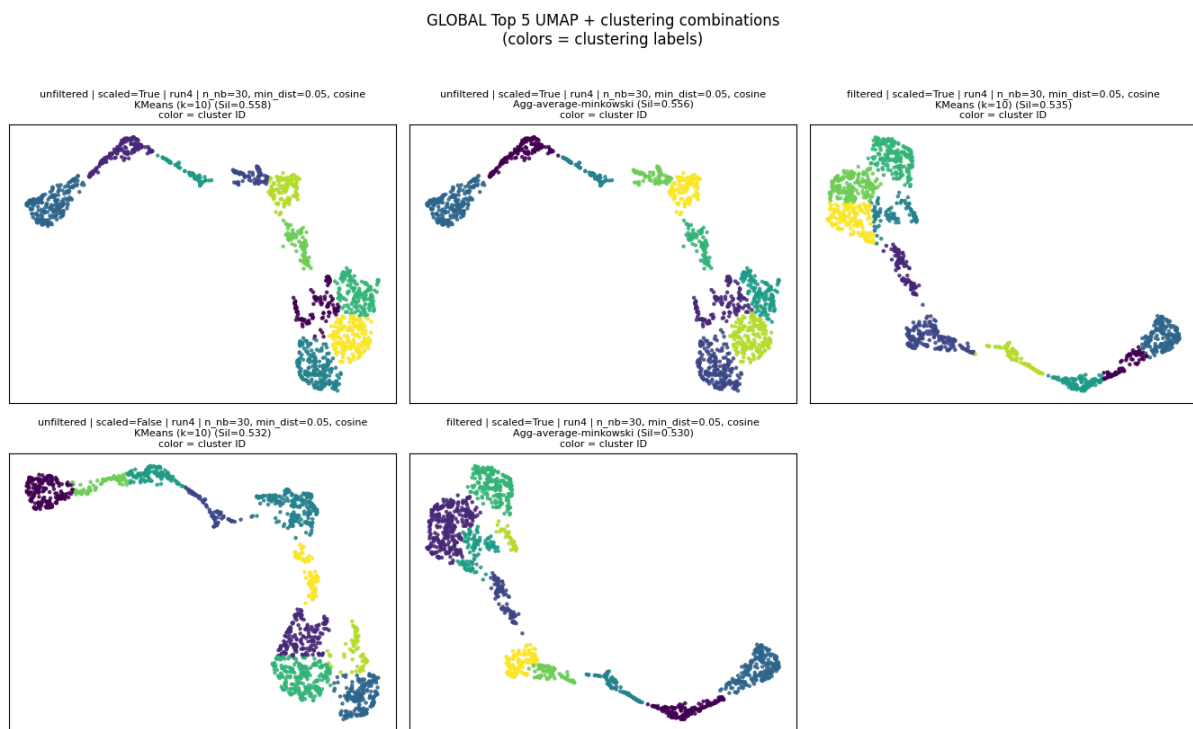
From Figure 2.4.4, which shows the UMAP embeddings colored by unsupervised cluster labels, we see that the cells form distinct groups arranged along a continuous

manifold. This indicates the presence of real underlying structure in the data. As with t-SNE, UMAP determines only the 2-dimensional layout, while the clustering algorithms assign the labels; both steps are entirely unsupervised and rely only on gene-expression similarity.

When the same embeddings are colored by the true biological cell types in Figure 2.4.5, it becomes clear that the annotated cell types do not align with the unsupervised clusters. Each cell type is spread across multiple regions of the manifold rather than forming a single compact group. This shows that, although UMAP captures meaningful variation, this variation does not correspond to the broad annotated cell types but instead reflects other continuous biological processes present in the dataset. The same conclusion was reached with t-SNE in the previous section.

Figure 2.4.4

UMAP embeddings for the top 5 configurations, colored by predicted cluster labels



UMAP embeddings for the top 5 configurations, colored by true biological cell types

Figure 2 displays six t-SNE plots showing cell clusters colored by cell type. The plots are arranged in a 2x3 grid. The top row shows results for 'scaled=True' and the bottom row for 'scaled=False'. The columns represent different clustering methods: KMeans (left), Agg-average-minkowski (middle), and KMeans (right). Each plot includes a legend for cell types: CMP_broad, GMP_broad, LMPP_broad, STMSC_broad, MEP_broad, MPV1_broad, MPV2_broad, MPV3_broad, STMSC_broad, and other. The plots show that scaling improves the separation of cell types, particularly for the STMSC_broad cluster.

The last non-linear method tried for dimensionality reduction is using autoencoders. Here too, this new method is applied to the standardized dataset, in order to see how the results will vary with the same clustering techniques, measured with the various distance metrics.

26

compressed representation, and the reconstruction error drives the training process. Because the network is trained to reproduce the input as faithfully as possible, the latent space tends to preserve the structural patterns that are essential for distinguishing different cell populations.

2.5.1 General implementation and testing of multiple latent dimensions

To guide the training of the autoencoder and quantify how well the model reconstructs the input from its latent representation, the reconstruction error is measured using mean squared error (MSE). This loss function evaluates the average squared difference between the original expression profile and its reconstructed version, allowing the network to adjust its parameters based on how accurately the latent space captures the underlying structure of the data. The optimisation process is carried out using the Adam optimiser, which combines adaptive learning rates with momentum-based updates, providing stable and efficient convergence during training.

Multiple latent dimensionalities were evaluated to understand how different levels of non-linear compression affect the structure of the embedding and the downstream clustering performance. In the implemented pipeline, the autoencoder is trained on a preprocessed dataset obtained through HVG selection and full z-score scaling, applied both to the unfiltered data and to the biologically filtered version. After training, the encoder produces latent representations whose dimensionality can vary substantially depending on the chosen bottleneck size.

Three configurations were tested: a latent space equal in size to the input (no effective compression), a moderately compressed space of 50 dimensions, and a strongly compressed space of 10 dimensions. Examining these different latent sizes has been considered relevant because each configuration interacts differently with the clustering algorithms that will be used: for example, a high-dimensional latent space preserves most of the original variability but risks retaining noise that reduces silhouette separation.

To resume, the same set of clustering and evaluation procedures used for the PCA analysis is applied again in the autoencoder setting, but separately for each latent dimensionality. For both the filtered and unfiltered datasets, every latent space produced by the autoencoder is clustered with all previously introduced methods and evaluated using the same silhouette-based metrics, allowing the effect of different levels of non-linear compression to be directly compared.

2.5.2 Overview of the top autoencoder-based results

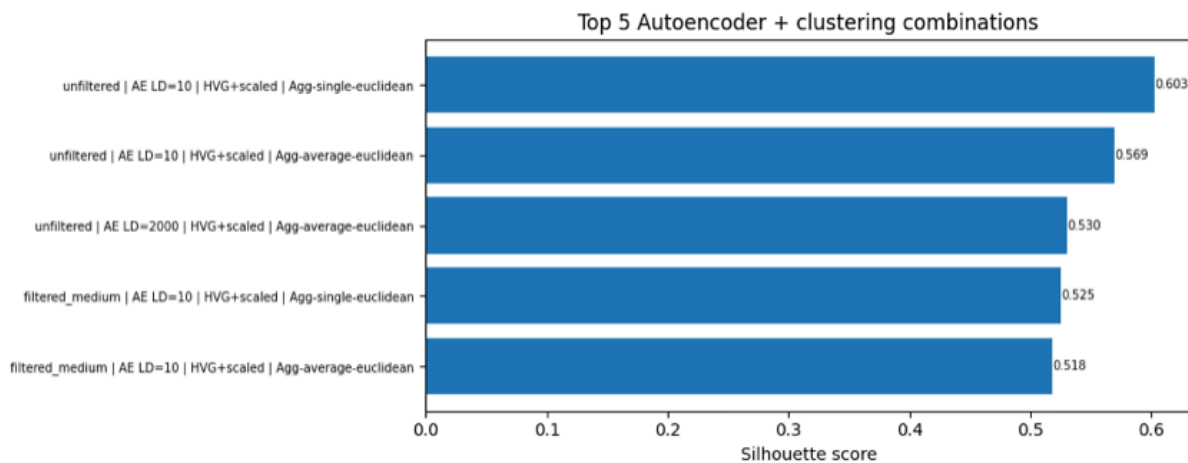
From the results in figure 2.5.1, it can be observed that all top configurations use the *HVG + scaling* pipeline and achieve silhouette scores between ~0.52 and ~0.60. The best combination is unfiltered, latent dimension=10, HVG+scaled, agg-single-euclidean, with a silhouette score of 0.603, indicating relatively compact and well-separated clusters in the 10-dimensional latent space. The configuration with average linkage on the same latent space

(agg-average-euclidean, 0.569) performs similarly, suggesting that the structure captured by the autoencoder is robust to the choice of hierarchical linkage.

The plot also highlights the effects of latent dimensionality and biological filtering. The model with a very large latent dimension (2000) still appears among the top results but with a slightly lower silhouette (0.530), consistent with the idea that an excessively high-dimensional latent space reintroduces noise and reduces cluster compactness. Likewise, the *unfiltered* datasets consistently outperform the filtered ones, indicating that, in this setting, removing biologically confounding genes is not necessary to obtain good geometric separation once HVG selection and scaling are applied.

Figure 2.5.1

Silhouette score for top 5 Autoencoders and clustering methods



3 Supervised modeling and evaluation

In this chapter we describe the supervised learning pipeline used to classify single cells into ten hematopoietic populations from their gene expression profiles. The pipeline combines a large combinatorial search over preprocessing strategies and model families with a nested cross-validation scheme that aims to provide an almost unbiased estimate of predictive performance.

Throughout the chapter we use macro-averaged F1 as the primary metric, because it weights rare and abundant cell types equally. Using this rigorous evaluation, the best pipeline, an ExtraTrees ensemble operating on PCA-compressed highly variable genes (HVGs), reached a macro-F1 of about 0.44 in nested cross-validation, far above naive baselines. The rest of the chapter details the methods (Section 3.1), the main quantitative results (Section 3.2), the final refit and ensemble attempts (Section 3.3), biological interpretation (Section 3.4), and a brief summary (Section 3.5).

3.1 Methods

3.1.1 Preprocessing and search space

Before selecting or training any classifier, we needed to construct informative, clean input representations from the sparse, high-dimensional single-cell RNA-seq data. Our preprocessing pipeline is built around highly variable gene (HVG) selection, biology-motivated gene filtering, optional removal of cell-cycle effects, different representation-learning strategies, and the possibility of adding handcrafted marker scores. These choices are treated as discrete factors that define a combinatorial search space of possible preprocessing pipelines.

HVG selection forms the backbone of the feature construction. For each gene we computed a dispersion statistic across cells and selected subsets of the most variable genes, under the assumption that they are most likely to capture biologically relevant heterogeneity. Across our experiments we allowed the number of HVGs to vary over the set $\{250, 1000, 1250, 1500, 1750, 2000, 4000\}$. This acts as a coarse control over input dimensionality: too few HVGs risk discarding important lineage-defining markers, while too many reintroduce noise and redundancy.

3.1.2 Biology-motivated gene filters.

On top of HVG selection we applied explicit gene-family filters that target known technical or broadly non-informative sources of variation as detailed in Section 1.6. The filters are implemented as masks on the gene axis, constructed from the gene symbols in `adata.var`:

Each filter level can be applied either before HVG selection (“pre-filtering”, where filtered genes are never allowed to become HVGs) or after HVG selection (“post-filtering”, where HVGs are chosen on the full set and then intersected with the filter mask). Pre- vs post-filtering are therefore separate categorical choices in the search space.

3.1.3 Cell-cycle regression (“clean_cc”).

In addition to removing genes, we implemented a regression-based variant that keeps the genes but attempts to subtract cell-cycle effects at the expression level. This “clean_cc” option is encoded as a special configuration: it uses the mild filter (removing ribosomal and mitochondrial genes), applied pre-HVG, and sets a dedicated `regress_cc` flag to true.

Technically, cell-cycle regression proceeds as follows within each training fold. First, we build two boolean masks over genes, corresponding to S-phase and G2/M-phase markers, by matching a curated list of symbols against the global gene list. For each cell we then compute two scalar covariates: the mean expression of S-phase markers and the mean expression of G2/M markers. These two scores, together with an intercept term, form a 3-column design matrix $Z=[1, "cc_S", "cc_G2M"]$. For the current training cells we solve a

least-squares problem $X \approx Z\beta$ in matrix form, where X is the (cells $\times$ genes) expression matrix restricted to the selected genes. The fitted coefficients β are stored and the residuals $X - Z\beta$ replace the original expression values. At transform time the same regression coefficients are applied to new cells, using their S and G2/M mean expressions, ensuring a consistent “cell-cycle-regressed” representation across training and validation folds.

3.1.4 Representation learning and marker scores.

Beyond gene-level selection, we explored several representation strategies. In the simplest case we use standardized HVG expressions directly (“HVG only”). Alternatively, we apply principal component analysis (PCA) to the HVG matrix, optionally with whitening so that PCs have unit variance, which is beneficial for scale-sensitive classifiers such as SVMs. The number of PCs is chosen from grids that depend on the HVG count. Finally, we tested autoencoder (AE) embeddings with latent dimensions between 8 and 64, trained on the HVG space to produce non-linear low-dimensional representations. In all cases, the representation model (PCA or AE) is fitted only on the training cells of the current fold and then reused to transform validation or test cells.

In some variants we append biologically curated marker scores to the learned representation. The marker sets are defined by the helper `get_population_marker_sets`, which assembles lists of canonical genes for broad hematopoietic populations. The base lists cover archetypal “*_like” groups such as HSC-like, ST-HSC-like, CMP-like, GMP-like, MEP-like, LMPP-like and so on, using markers from the literature and from our earlier exploratory analysis. From these, additional “broad” populations are constructed by unioning related sets (for example, a broad LTHSC group pooling HSC-like and ST-HSC-like markers) and by forming heuristic combinations for MPP-like states (e.g. myeloid-biased, erythroid-biased or lymphoid-biased mixes of CMP / GMP / MEP / LMPP markers). All gene symbols are converted to upper case and deduplicated.

Given these marker definitions, the transformer maps each marker set to indices in `adata.var`. For any collection of cells (training or validation), the marker score for a given population is simply the mean expression of all genes in its marker list, computed per cell. This produces a matrix of shape (cells $\times$ number of marker sets). During fitting, if `use_marker_scores` is enabled, the transformer computes marker scores on the training cells, estimates per-marker mean and standard deviation, and uses these statistics to standardize marker scores for both training and validation cells. The standardized marker scores are then concatenated to the main representation (HVGs, PCs or AE codes), providing the classifier with a small set of interpretable, biology-aware features alongside the automatically learned ones.

All of the above choices can be summarized as a 6-tuple:
 ("filter_stage", "filter_level", "regress_cc", "HVG_count", "representation", "use_marker_scores")
 which defines a single preprocessing variant. Concretely:
 $\text{"filter_stage"} \in \{\text{"pre"}, \text{"post"}\}$

`"filter_level" ∈ {"none", "mild", "medium", "aggressive"}`
`"regress_cc" ∈ {"False", "True"}`, with the special “clean_cc” variant corresponding to
(stage = pre, level = mild, regress_cc = True)
`"HVG_count" ∈ {250, 1000, 1250, 1500, 1750, 2000, 4000}`
`"representation" ∈ {"HVG", "PCA"(k), "PCA_whiten"(k), "AE"(d)}`, where k is chosen
from a small grid of PC counts that depends on the HVG count and from a grid of AE latent
dimensions.
`"use_marker_scores" ∈ {"False", "True"}`.

For convenience, the three biology-related components
(`"filter_stage"`, `"filter_level"`, `"regress_cc"`) are pre-enumerated into a set of nine “bio specs”:
the eight combinations of stage × level with no regression, plus the special clean_cc spec.
Together with the seven HVG counts, the representation options (HVG-only, several PCA /
PCA-whiten grids and five AE latent dimensions per HVG choice) and the marker-score
toggle, this yields a large but finite catalog of preprocessing variants. Each supervised trial
samples one such 6-tuple and couples it with a model family and a specific hyperparameter
setting from that family’s discrete grid.

The full Cartesian product of preprocessing choices and model grids would amount to
roughly 4.3 million distinct pipelines. Computational constraints forced us to evaluate only a
tiny fraction of them keeping 15000 as maximum number of fits, which equated to 468
configurations (fit budget divided by 8 outer folds times 4 inner folds) , so we used a
pseudo-random “least-represented first” sampler that preferentially draws under-explored
options in each discrete dimension, keeping the marginal usage of HVG counts, bio specs,
representations and model families approximately balanced across trials.

3.1.6 Model families and hyperparameter grids

On top of the preprocessing choices, we evaluated a diverse panel of supervised
learning algorithms, deliberately including at least one representative of each major model
family: linear models, support vector machines, tree-based ensembles, neural networks,
gradient-boosted trees and a simple probabilistic baseline. Each classifier is wrapped in a
pipeline where the HVG/representation transformer provides the input features and the
classifier forms the final prediction step.

For logistic regression we considered both L1 (sparse) and L2 (ridge) penalties with
inverse-regularization strengths C in the range 0.25-4.0. Linear SVMs share the same set of C
values and differ mainly in their hinge-loss objective, which emphasises large margins. For
both families we optionally enabled `class_weight="balanced"`, although this is partly
redundant with the sample weighting described below.
Non-linear SVMs with RBF kernels were tuned over the same C values together with a range
of kernel widths γ (including the standard “scale” and “auto” settings as well as fixed
numerical values). These models operate on PCA- or AE-based representations with
standardized features to keep the kernel well behaved.

We used multilayer perceptrons (MLPs) implemented in a custom PyTorch classifier depending on GPU availability. The search space covers a variety of architectures, from shallow single-layer networks with 64-512 hidden units to deeper networks with two to four layers. Hyperparameters include the activation function (ReLU or tanh), dropout rate (0-0.3), L2 weight decay (10^{-6} to 10^{-4}) and learning rate (3×10^{-4} to 10^{-3}). All MLPs were trained with early stopping based on the training loss to prevent overfitting.

For tree-based ensembles we used Random Forests and Extremely Randomized Trees (ExtraTrees). Their grids include the number of trees (200-1200), maximum depth (10, 20, 40), the minimum number of samples required to split a node (2, 5, 10) and the feature subsampling scheme ("sqrt", "log2" or a fixed fraction of features per split). Gradient-boosted trees are represented by XGBoost models, tuned over the number of boosting rounds, tree depth, learning rate, subsampling fraction, feature subsampling and ℓ_2 regularization strength. Finally, a Gaussian Naive Bayes classifier serves as a simple baseline with a small grid over the variance-smoothing parameter.

In all cases, the classifier is the only step in the pipeline that sees class labels, while all preprocessing is purely unsupervised. During training we pass per-sample weights to any estimator that supports them so that each class contributes approximately equally to the loss, matching the macro-F1 objective used for model selection.

3.1.7 Nested cross-validation and evaluation

To obtain a defensible estimate of how well our model-selection procedure would generalize to new data, while exploring a very large space of pipelines, we used a nested cross-validation (CV) scheme with a fixed list of trial configurations (Figure 3.1).

We first define a list of about $K \approx 468$ trials, called $T_1, \dots, T_K$.

Each trial is a *frozen pipeline*: one specific choice of

- preprocessing 6-tuple (filter stage, filter level, cell-cycle handling, HVG count, representation type and size, marker scores on/off),
- model family (e.g. ExtraTrees, Logistic Regression, MLP, ...), and
- concrete hyperparameters for that model.

These configurations are sampled once from the combinatorial space described in Section 3.1., before any cross-validation starts. During nested CV nothing inside a trial is tuned: the inner CV only evaluates how well each frozen pipeline performs when it is refitted on different splits of the data.

To evaluate the selection procedure, the outer loop uses $K_{\text{outer}} = 8$ stratified folds on the labeled dataset D .

For each outer fold $f = 1, \dots, 8$ we split D into:

- outer-train(f): 7/8 of the labeled cells, used for training and inner CV in this fold;

- outer-test(f): the remaining 1/8 of the labeled cells, held out as the test set for fold f.

The outer-test(f) is never used for feature learning, model fitting, or choosing between trials. It is touched exactly once, at the end of fold f, to evaluate whichever trial is selected for that fold. Within outer-train(f) we now evaluate all K trials using an inner 4-fold CV.

The inner loop works as a stress test for frozen trials on the outer-train.

Fix an outer fold f and a particular trial T_i . We run a 4-fold inner CV on outer-train(f):

1. outer-train(f) is partitioned into four inner splits
(inner-train(f,1), inner-valid(f,1)), ..., (inner-train(f,4), inner-valid(f,4)).
2. For each inner split $j = 1, \dots, 4$ we fit the entire pipeline T_i from scratch on inner-train(f,j):
 - select HVGs using only inner-train(f,j);
 - apply the biological filters (pre or post) on that training set only
 - fit PCA or the autoencoder on inner-train(f,j), if that representation is used;
 - fit the classifier parameters (trees, weights, network weights, ...) on inner-train(f,j).
3. We then apply this fitted preprocessing and classifier to inner-valid(f,j) and compute macro-F1.

Averaging the four inner-fold F1 scores gives a single mean inner F1 in fold f for trial T_i , which we denote informally as $\text{mean_inner_F1}(f, T_i)$. Conceptually, the inner loop is a stress test of a fixed recipe: we only change the training/validation partition of the data; we never change the hyperparameters or preprocessing choices inside a trial.

Figure 3.1 (yellow callouts) highlights two key properties:

- No feature leakage. HVGs, filters, PCA, autoencoders, marker scores, etc. are always fitted on inner-train (or on outer-train at the final refit stage). inner-valid and outer-test cells are never used to compute representations.
- No “lucky draw”. Each trial must perform reasonably well across four different inner splits; we do not trust a single convenient train/validation split.

Still within outer fold f, once we have $\text{mean_inner_F1}(f, T_i)$ for all trials $i = 1, \dots, K$, we:

1. Select the best trial for this outer fold.
We choose $T^*(f)$ as the trial with the highest $\text{mean_inner_F1}(f, T_i)$ across all i .
2. Refit this chosen trial once on all of outer-train(f).
We recompute HVGs, filters, PCA/AE and retrain the classifier using the full 7/8 training portion for fold f.
3. Evaluate this refitted pipeline on outer-test(f) to obtain an outer F1(f) (and accuracy) for fold f.

No other trial is ever applied to outer-test(f); all other pipelines are used only inside inner CV on outer-train(f). Repeating this process for all eight outer folds produces eight outer F1

scores. By concatenating the predictions for all outer-test folds we obtain the stitched outer F1 that can be used as a proxy for generalization.

It is crucial to understand what this stitched outer performance represents: it is an almost unbiased estimate of the behaviour of the entire selection procedure:

“run inner CV over all trials → pick the best by inner F1 → refit on outer-train
→ test on outer-test”

It is not the performance of any single fixed model, as by design these 8 F1 scores might come from different Trials. We never use these outer scores to choose the final model; they are purely a sanity check that the selection strategy itself generalizes well and would likely work on a completely new dataset.

In parallel, we still want to produce an aggregated inner score and find a Global Best Trial. The inner loop provides a performance profile for each individual trial across folds. For every trial T_i we collect its $\text{mean_inner_F1}(f, T_i)$ over all outer folds f and compute a global mean inner F1 for T_i . This tells us how robustly that frozen pipeline performs across many different resamplings of the training data, always evaluated on held-out inner-validation splits.

After nested CV is complete we:

1. Rank all trials T_i by their global mean inner F1.
2. Choose a single Global Best trial T^* (the top-ranked pipeline; in our case this turns out to be an ExtraTrees-based pipeline).
3. Refit T^* once on the full labeled dataset D and use this final model to predict the previously unlabeled target cells (Section 3.3). These predictions are used only for inference on unlabeled cells; for our reporting purposes, a confusion matrix of the performance of this model on a simple 8 fold cross validation on the available dataset is shown in Figure 3.12

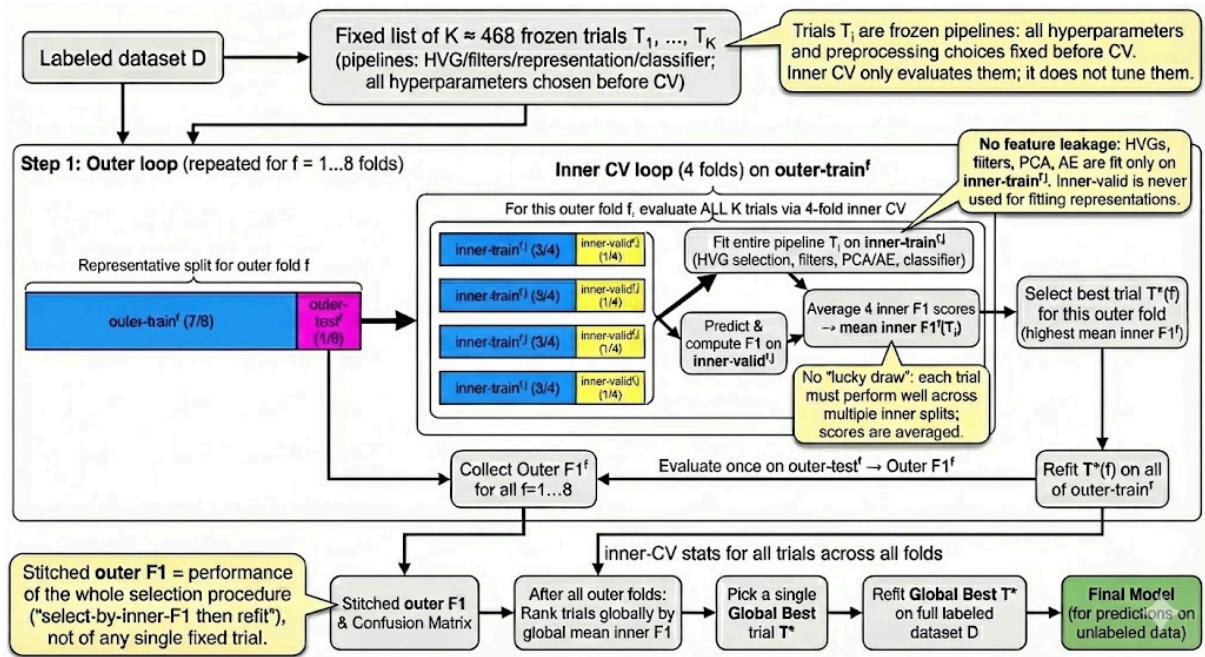
In summary, the nested scheme cleanly separates two goals:

- The outer loop evaluates how well our *model-selection procedure* performs on unseen data (stitched outer F1).
- The inner loop summarizes which frozen pipelines are intrinsically strong and supports the final choice of a single Global Best trial.

Figure 3.1 illustrates this data flow step by step: each frozen trial is evaluated by a 4-fold inner CV within every outer-training split; the best trial per outer fold is refit and tested on that fold’s outer-test set; and, after all folds are complete, one Global Best trial is selected based on aggregated inner-CV performance and retrained on all labeled cells for final predictions.

Figure 3.1

Data flow of the nested cross-validation pipeline.



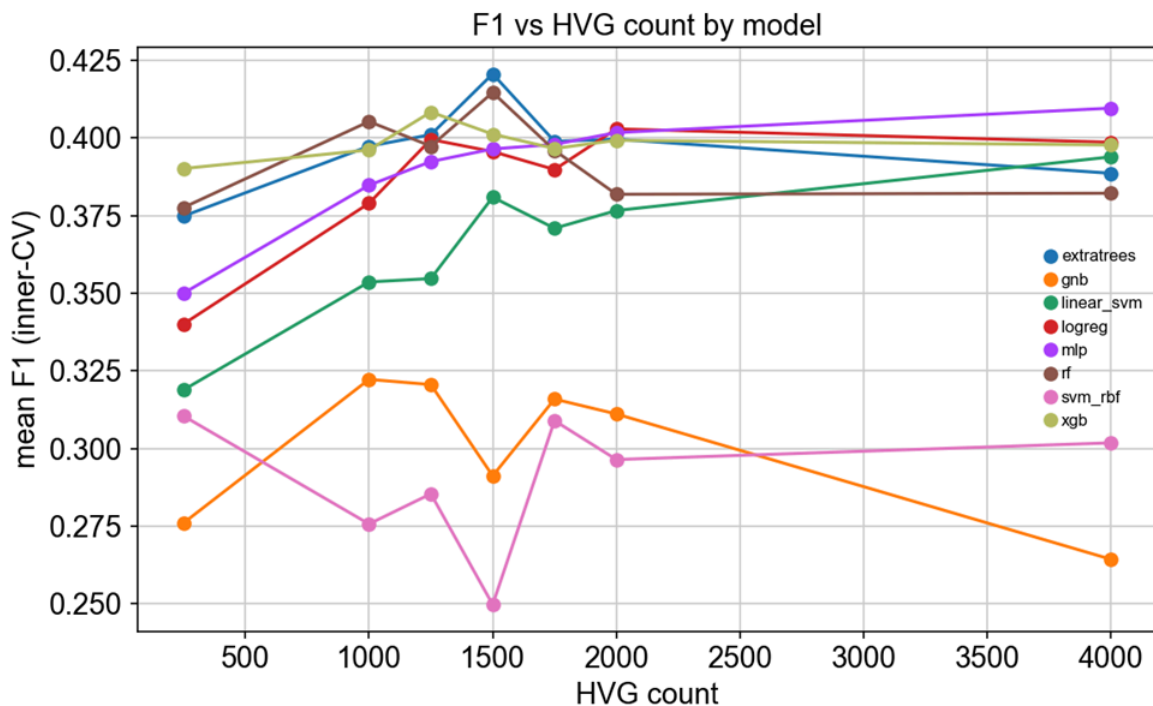
3.2 Results

3.2.1 Effects of preprocessing choices

We first analyse how the preprocessing parameters influence performance, marginalising over model families. The most prominent effect comes from the number of HVGs. Figure 3.2 shows the mean macro-F1 across all trials as a function of HVG count. Pipelines using only 250 HVGs consistently underperform, reflecting the loss of discriminative markers. Performance improves sharply when moving from 250 to around 1500 HVGs and then plateaus between 1500 and 2000, with only a mild decline at 4000 genes where more noise and redundant features are reintroduced, this was our proxy for a lighter preprocessing cut through HVG. Notably, the final best models all use HVG counts in this 1500-2000 sweet spot.

Figure 3.2

Mean macro-F1 across trials grouped by number of HVGs. No

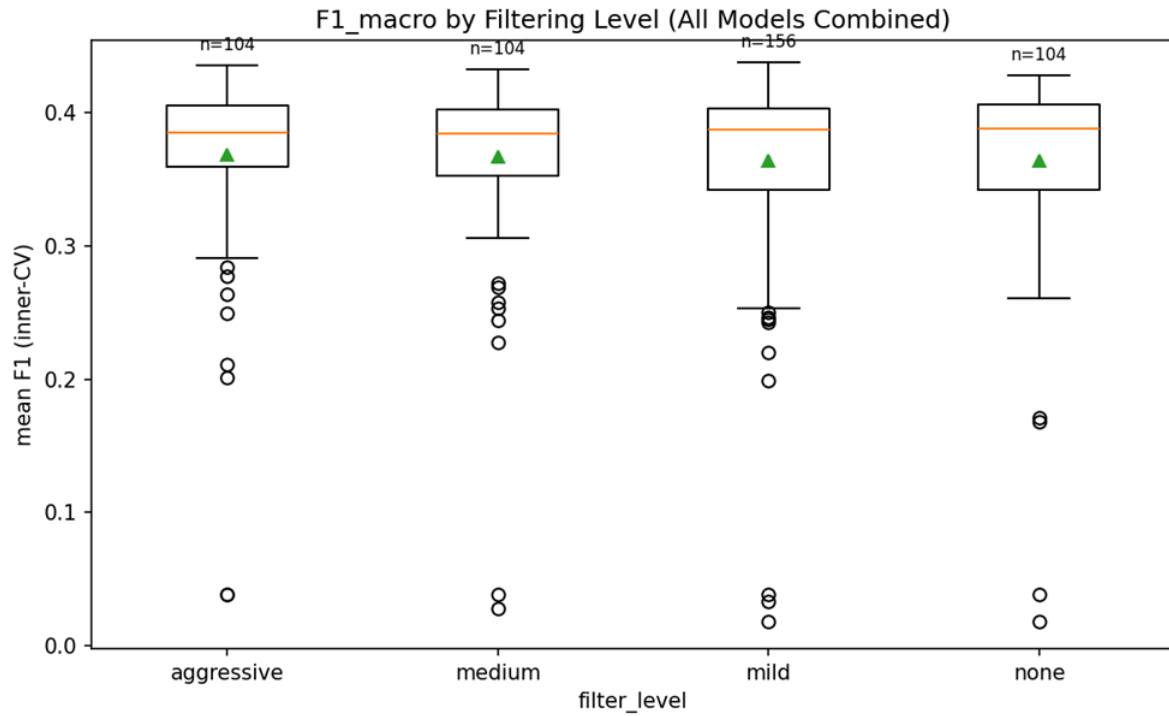


Note: Performance rises steeply from 250 to around 1500 HVGs, peaks near 1500-2000 and slightly declines or remains stable at 4000 genes.

Gene filtering turned out to have a surprisingly modest impact. As summarised in Figure 3.3, the distributions of macro-F1 are remarkably similar for the four filter levels (none, mild, medium, aggressive). Medium filtering, which removes ribosomal, mitochondrial, immunoglobulin and hemoglobin genes, was initially expected to be clearly superior, but in practice all strategies achieve comparable median performance. Aggressive filtering introduces slightly more variability, occasionally hurting models that would have benefited from keeping some of the removed genes.

Figure 3.3

Distribution of macro-F1 scores by gene filtering strategy (none, mild, medium, aggressive) aggregated over all model families.

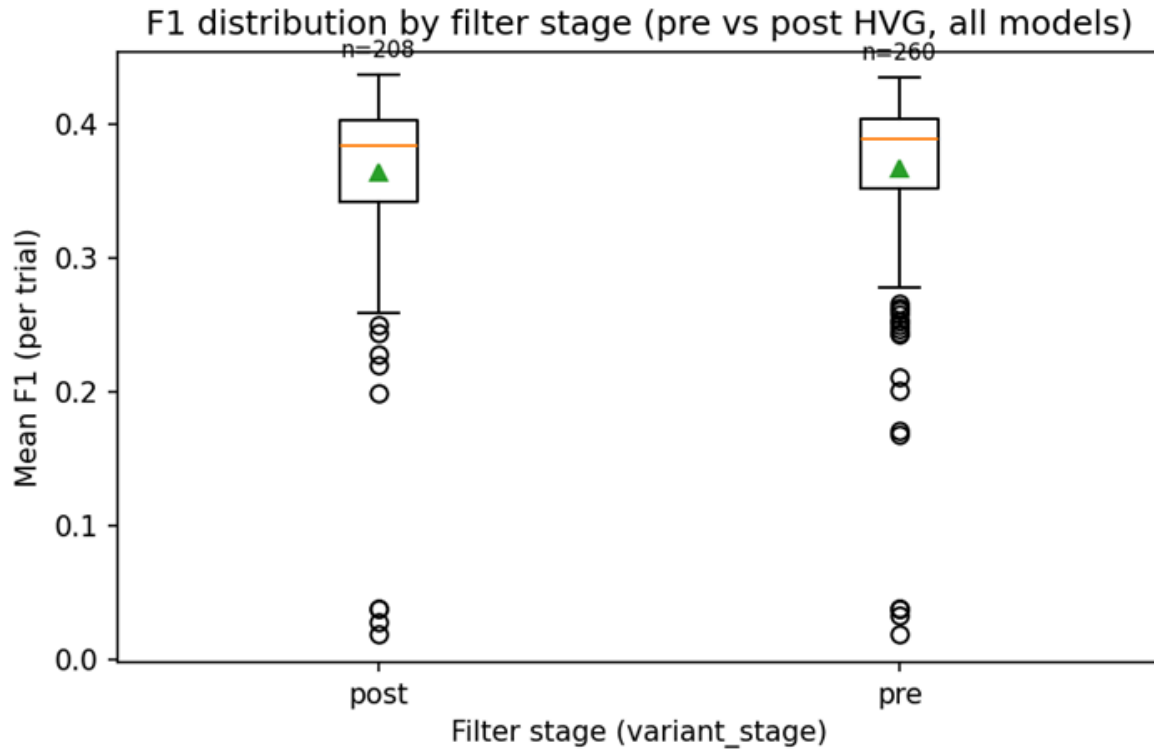


Note: Median performance is similar across levels, with aggressive filtering showing slightly higher variance.

A small but consistent advantage is observed for pre-filtering over post-filtering: when filters are applied before HVG selection we avoid wasting HVG slots on genes that will later be discarded, such as highly variable immunoglobulin chains, as shown in Figure 3.4.

Figure 3.4

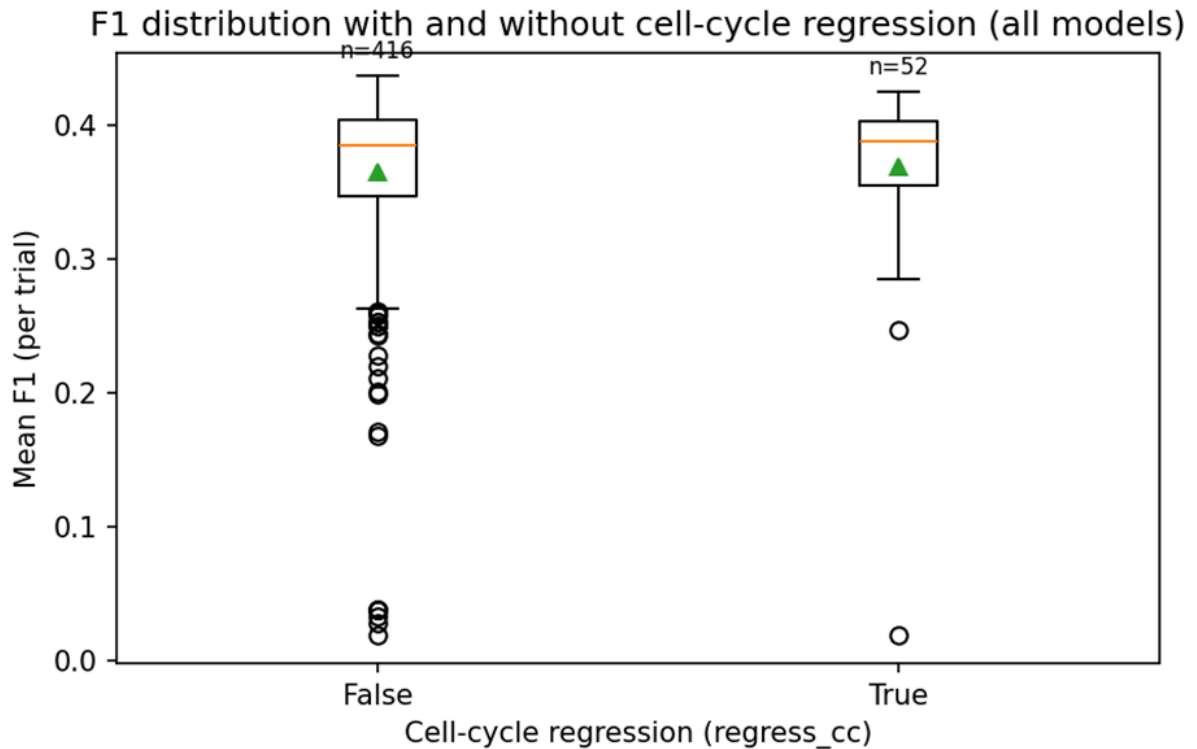
Distribution of macro-F1 scores by gene filtering stage (Pre/Post HVG) aggregated over all model families.



The “clean_cc” variant regresses out cell-cycle effects while retaining all genes. Although conceptually appealing, this strategy does not deliver a clear benefit over medium filtering in our experiments. The nested-CV results suggest that, for this classification task, simply excluding the most cell-cycle-dominated genes is enough; additional regression brings only marginal gains, if any. These effects are summarized in Figure 3.5

Figure 3.5

Distribution of macro-F1 scores by usage of the cell cycle regression filter, aggregated over all model families.

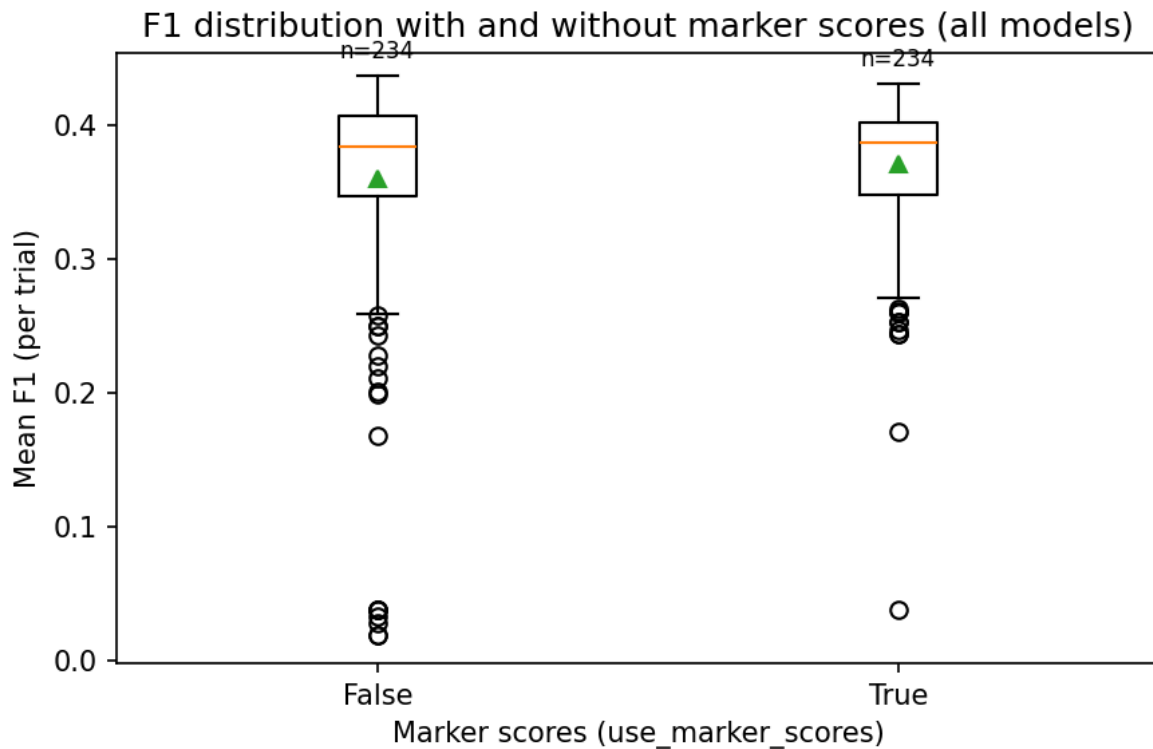


Note: Median performance does not change too much other than showing a bigger set of lower-bound outliers for when the filter is not used.

Finally, we tested augmenting the representations with marker scores derived from curated gene sets for each broad population. These additional features are highly interpretable and in principle encode lineage programmes captured in prior biological knowledge. Empirically, however, marker scores only rarely improved macro-F1, as shown in Figure 3.6, and only one of the top-ranking pipelines used them. This suggests that PCA or AE applied to a sufficiently rich HVG set already reconstructs the major lineage axes, making explicit marker aggregates partly redundant.

Figure 3.6

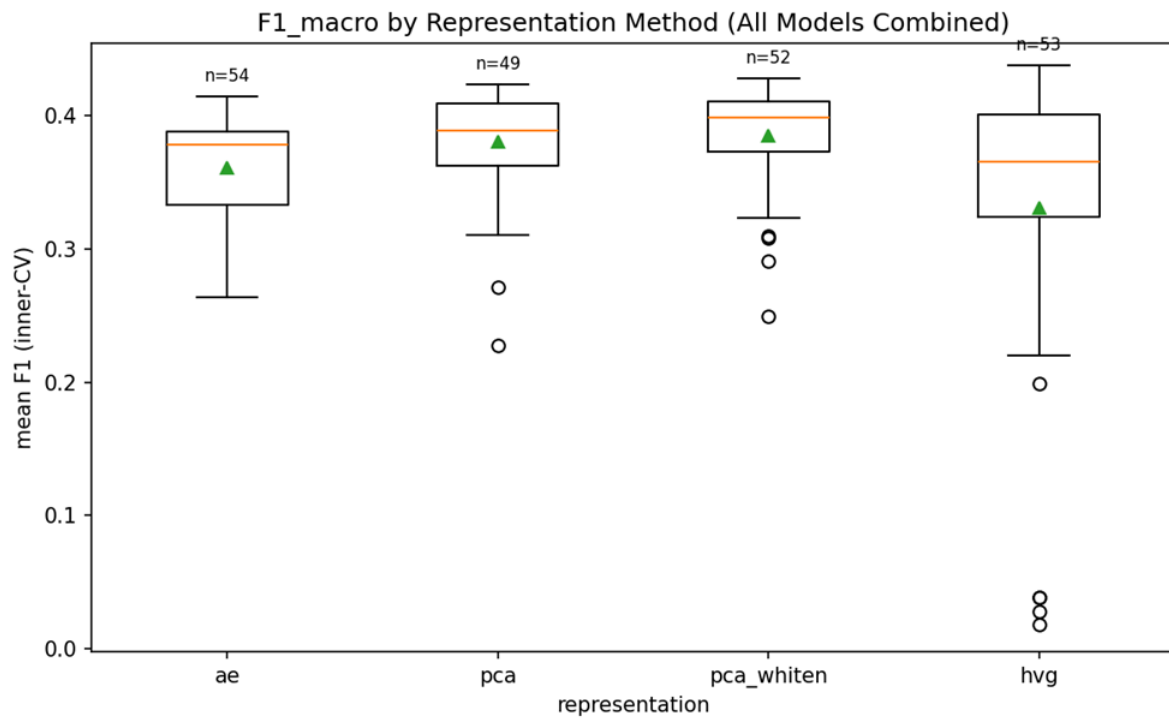
Distribution of macro-F1 scores by usage of augmented marker scores, aggregated over all model families.



Representation learning strategies also show systematic patterns. Figure 3.6 compares performance grouped by representation type. Using scaled HVG expression directly works reasonably well, but PCA-based representations are over-represented among the top-performing pipelines. Whitening the PCs gives a modest boost for scale-sensitive models such as linear SVMs and logistic regression. Autoencoder embeddings can be competitive for some configurations but tend to be more fragile: their performance depends strongly on the exact architecture and training setup, and they are less frequently selected as the best trial within an outer fold.

Figure 3.7

Model performance grouped by representation method (raw HVGs, PCA, PCA-whitened PCA, autoencoders).

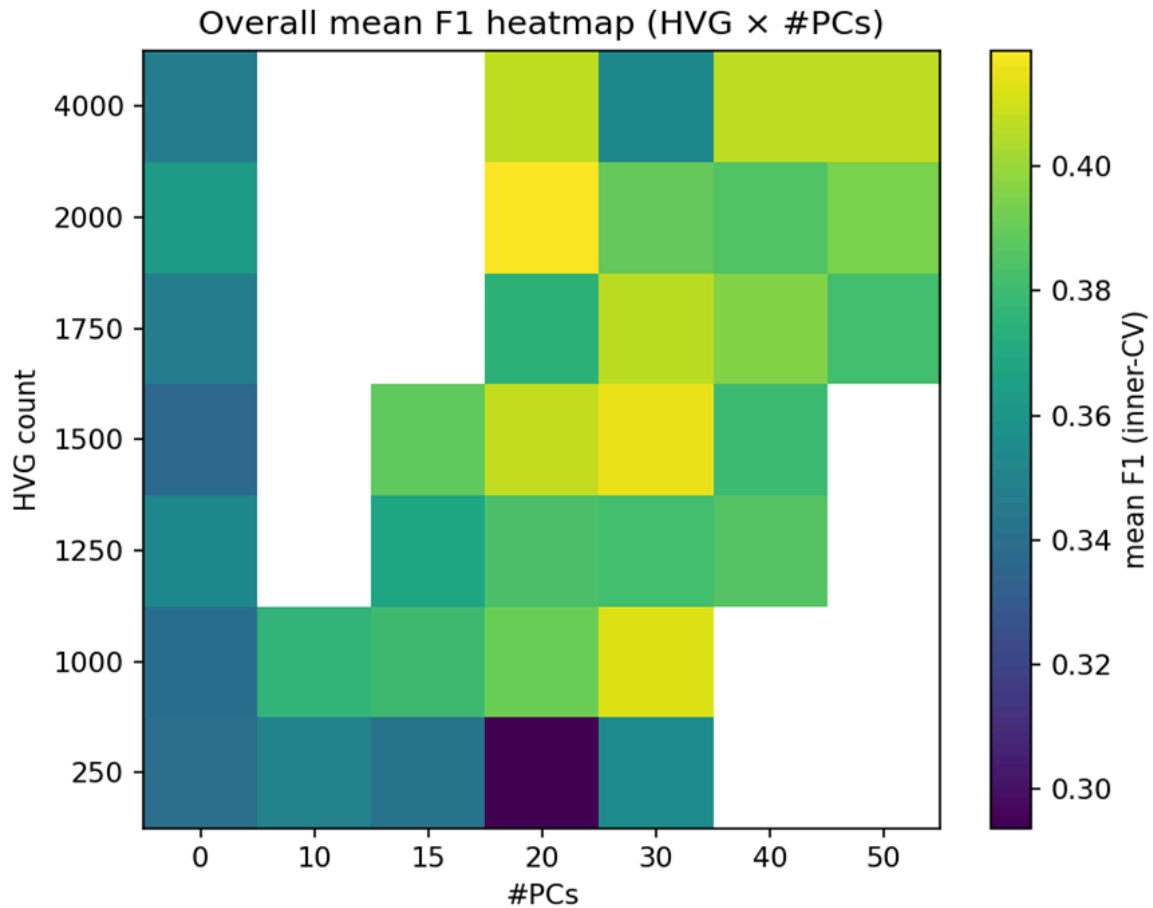


Note: PCA-based representations dominate among the best pipelines.

The joint effect of HVG count and number of PCs is summarized in Figure 3.8, which displays a heatmap of mean F1 across the two parameters. A clear ridge appears around 1500-2000 HVGs and 20-30 PCs, validating this range as the optimal trade-off between information retention and noise reduction.

Figure 3.8

Heatmap of mean macro-F1 across combinations of HVG count and number of principal components.



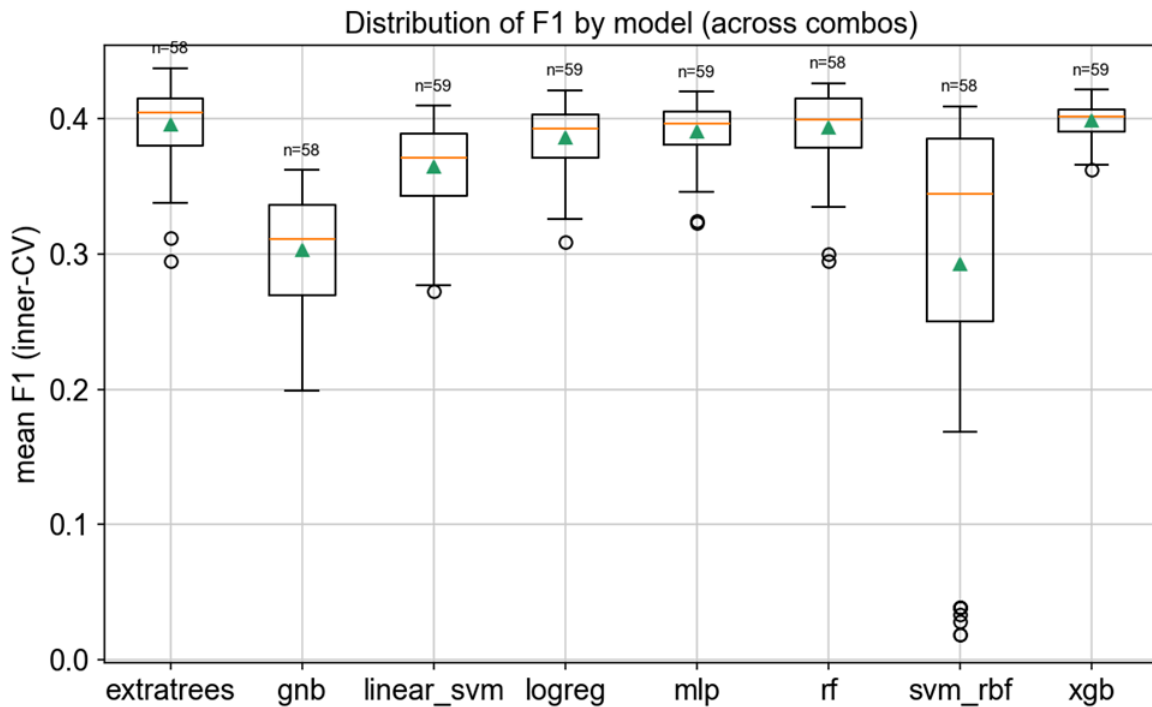
Note: A pronounced ridge appears around 1500-2000 HVGs and 20-30 PCs.

3.2.2 Model family comparison and best configurations

We now turn to the behaviour of the different classifier families within the nested-CV framework. Figure 3.9 shows boxplots of macro-F1 scores grouped by model type, each box summarising all trials of that family across inner folds. ExtraTrees and Random Forests occupy the top tier: they achieve the highest medians and produce most of the best-performing trials. Linear models (logistic regression and linear SVM) are surprisingly competitive, often reaching within a few percentage points of the best tree ensembles when paired with suitable PCA-compressed features, but slightly worse than MLPs. Kernel SVMs with RBF kernels and Gaussian Naive Bayes lag behind overall, confirming that complex non-linear kernels and overly simplistic independence assumptions are not ideal for this problem.

Figure 3.9

Boxplot of macro-F1 scores by model family. Note



Note: ExtraTrees and Random Forests show the highest median and top-end performance; logistic regression and linear SVMs are competitive when paired with PCA features;; RBF SVMs and GaussianNB perform notably worse.

The best nested-CV configuration is an ExtraTrees ensemble operating on a medium-filtered, pre-filtered gene set with 1750 HVGs and 20 principal components. The classifier uses 400 trees with maximum depth 40 and minimum split size 5, and no marker-score augmentation or cell-cycle regression. Averaged over the eight outer folds this pipeline achieves a macro-F1 of approximately 0.44 with an accuracy of around 62%. Considering that the task involves ten classes with substantial class imbalance and biologically subtle boundaries, this represents a strong result; random or majority baselines would obtain macro-F1 close to 0.1 or worse.

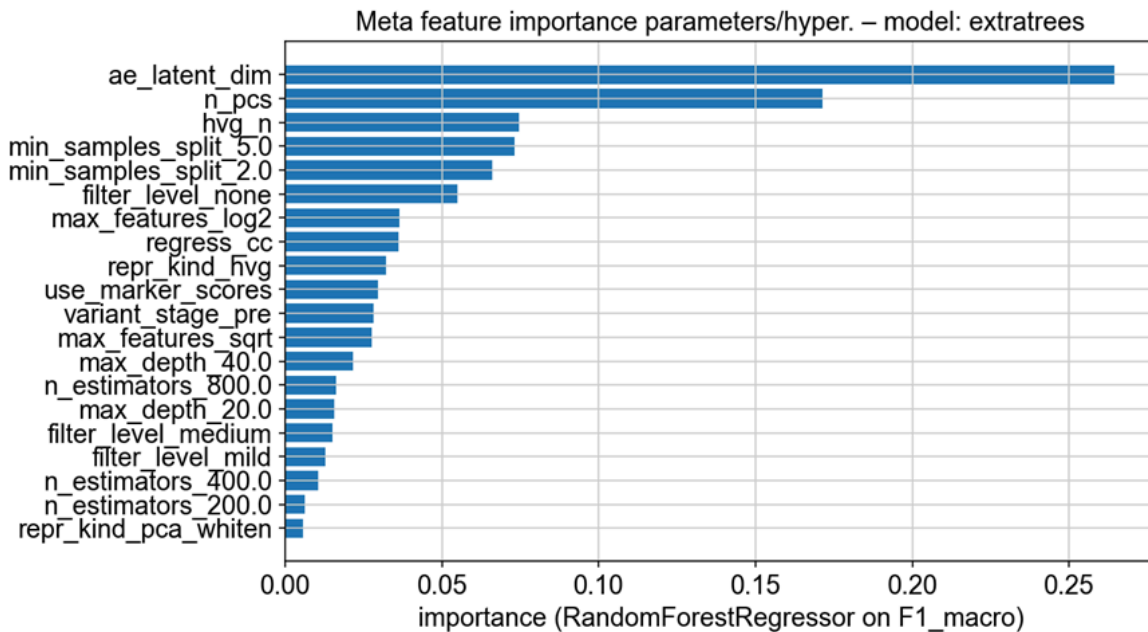
3.2.3 Hyperparameter importance meta-analysis

Given the large number of trials, we performed a meta-analysis to identify which design choices and hyperparameters most strongly influence model performance. For each model family we trained a RandomForestRegressor that predicts the macro-F1 of a trial from its preprocessing and hyperparameter settings (encoded as numerical values and one-hot indicators). The resulting feature importance scores quantify how much each parameter contributes to explaining variability in F1 across the random search. Across families a consistent pattern emerges: data-related parameters dominate over algorithm-specific knobs.

For tree-based ensembles, representation and dimensionality are still important but interact with tree-specific hyperparameters. In the ExtraTrees analysis (Figure 3.10), the strongest individual factor is whether an autoencoder representation is used at all; trials relying on AE features generally underperform those using PCA. The number of PCs and HVG count also play a role, although trees can tolerate somewhat higher dimensionality than linear models. Among classifier hyperparameters, the minimum number of samples required to split a node stands out: slightly larger values (for example 5 or 10) improve generalisation by preventing very deep, sample-poor splits that overfit noise. The number of trees and maximum depth are comparatively less critical within the ranges we explored.

Figure 3.10

Hyperparameter importance for ExtraTrees trials obtained from a RandomForestRegressor meta-model.

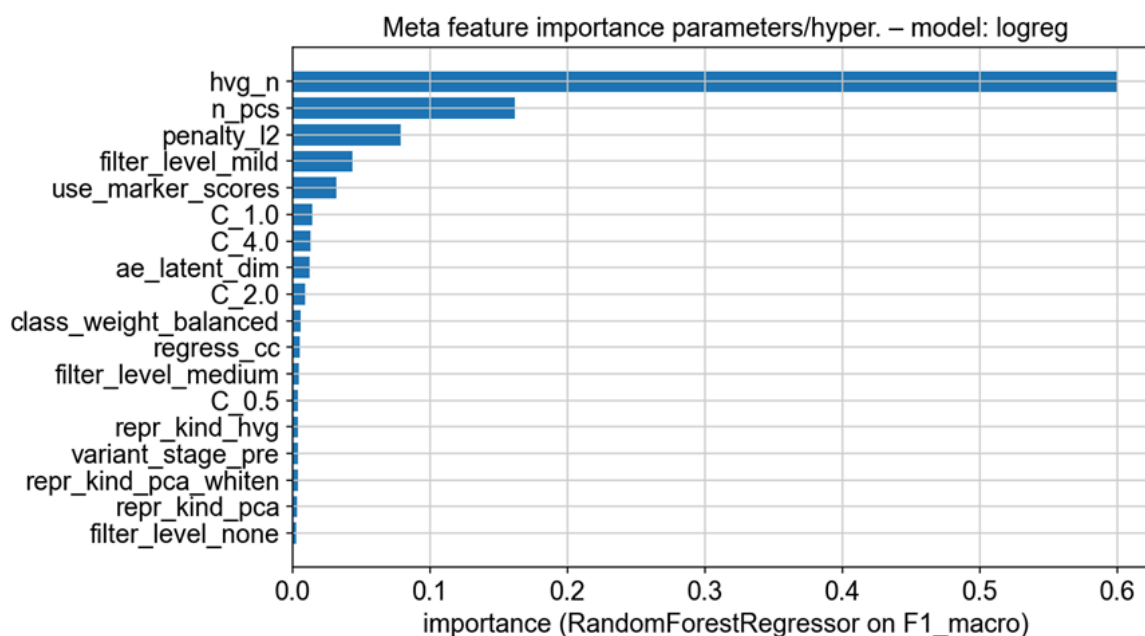


Note: Representation choices (autoencoder flag, number of PCs, HVG count) and the minimum samples per split have the largest influence on performance.

For logistic regression, illustrated in Figure 3.11, the number of HVGs and the number of principal components together explain the majority of performance variability. Choosing too few genes or too few PCs leads to a substantial drop in F1, whereas models using 1500-4000 HVGs and roughly 20-50 PCs tend to perform best. In contrast, the choice between L1 and L2 penalty or the presence of explicit class-weighting has only a modest effect.

Figure 3.11

Hyperparameter importance for logistic regression trials.No



Note: The number of HVGs and principal components dominate, whereas the choice of penalty (L1 vs L2) and class-weighting have relatively minor impact.

Similar behaviour is observed for MLPs, where HVG count and PC number again rank highest, while architectural details such as the exact hidden-layer sizes or learning rate matter less as far as the search space we have defined goes.

Overall, the hyperparameter importance analysis reinforces the message that, in this single-cell classification problem, “data first, model second” is a sensible strategy. Carefully choosing which genes and how many latent dimensions to feed into the classifier yields larger gains than fine-tuning the internals of a particular algorithm. Many different model families can approach the same performance ceiling if provided with a good representation of the data.

3.3 Final model, ensembles, and prediction pipeline

3.3.1 Final model refit and predictions

After identifying the best-performing configuration in nested CV we refit that exact pipeline on the full set of labeled cells to obtain the model used for downstream predictions. Concretely, we take the Global Best trial, the ExtraTrees pipeline described in Section 3.2.2

with its specific preprocessing 6-tuple and classifier hyperparameters, and apply the same fold-aware HVG selection, filtering, PCA and scaling to all labeled cells. The ExtraTrees classifier is then trained once on this full training set using class-balanced sample weights. Because this refit benefits from 100% rather than roughly 80% of the data available in each outer fold, its true performance on new data is expected to be slightly higher than the nested-CV estimate.

The refitted model is finally applied to the 500 unlabeled target cells. For each cell we obtain class probabilities over the ten populations, the predicted class (the most probable label) and a confidence score given by the maximum probability. These predictions form the basis of our submitted clustering labels. High-confidence predictions suggest well-resolved cell identities, whereas low-confidence assignments often coincide with cells located near class boundaries in the learned feature space.

3.3.2 Ensemble experiments

We also explored whether combining several strong models into an ensemble could further improve performance. Using the ranking produced by the aggregated inner-CV analysis we selected the top trials and built soft-voting ensembles that average class probabilities across models. Specifically, we considered an ensemble of the top 5 pipelines and another ensemble of the top 10, mixing ExtraTrees, Random Forests, logistic regression and MLP configurations with diverse preprocessing settings.

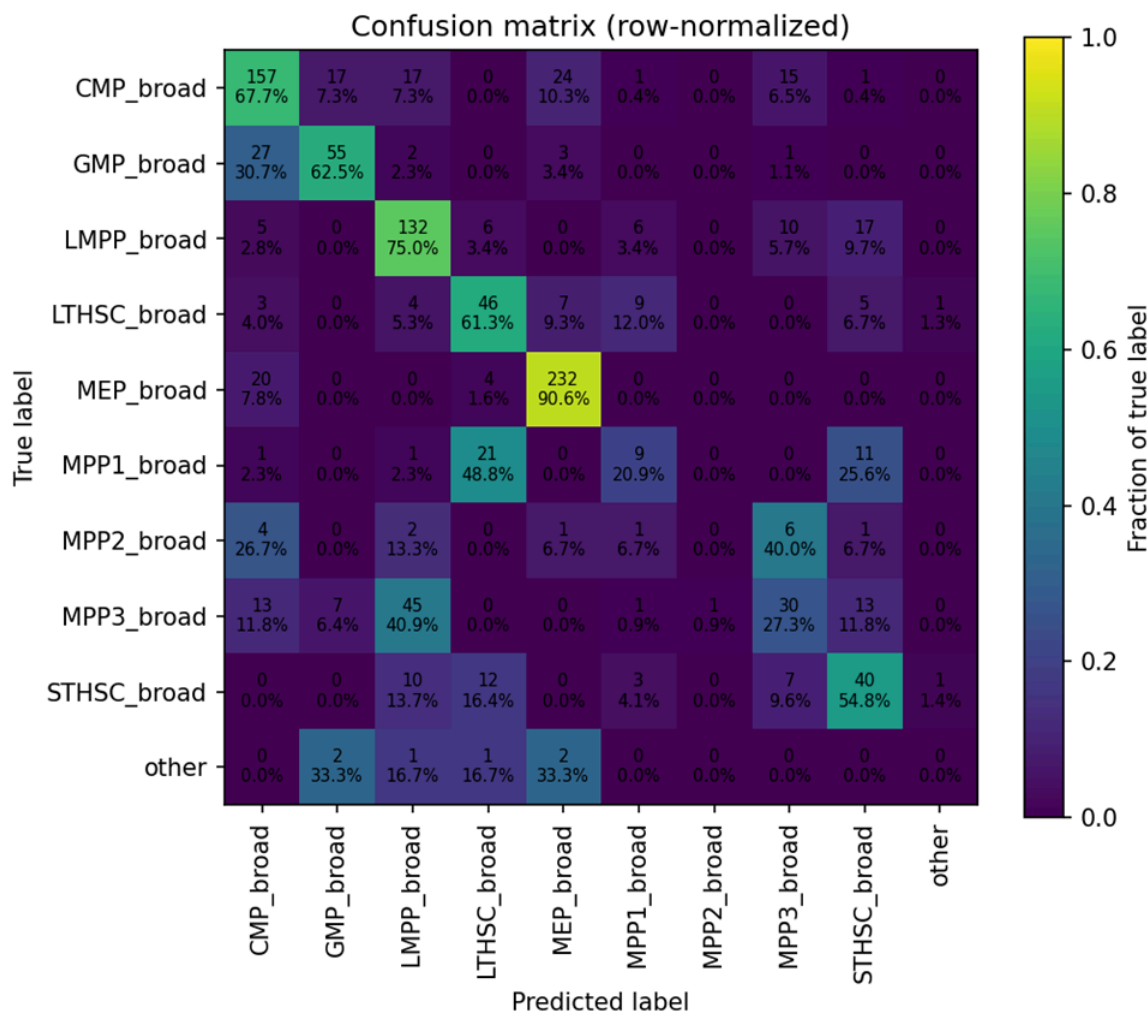
Each ensemble member is refitted on the full labeled dataset using its own preprocessing variant, after which their predicted probability vectors for each cell are averaged. These ensembles exhibit slightly higher scores on the hidden testset than any single model and, based on nested-CV behaviour, are expected to be at least as robust as the best individual pipeline. However, the gains over the single ExtraTrees model appear modest, and the increased computational cost and reduced interpretability led us to keep the single best pipeline as the primary model, with ensembles considered as a secondary robustness check.

3.4 Biological implications and error analysis

The control CV confusion matrix (Figure 3.12) shows that most errors occur between progenitor populations that are closely related. In the early HSPC compartment, long-term and short-term HSCs, LMPP and the MPP subsets are frequently confused with one another, reflecting their gradual developmental transitions and overlapping expression programs. Likewise, CMP, GMP and MEP mainly misclassify within this myeloid block rather than into unrelated classes.

Figure 3.12

Row-normalised confusion matrix of the final ExtraTrees pipeline.



Note: The confusion matrix is obtained by aggregating predictions across a normal CV performed at the end of the nested-CV procedure used to identify the best models. Misclassifications are concentrated among closely related progenitor populations, whereas more divergent lineages are well separated.

In contrast, lineages that diverge earlier in hematopoiesis, in particular lymphoid-primed LMPP versus the myeloid progenitors (CMP/GMP/MEP), are rarely confused. This pattern indicates that, although the classifier is purely data-driven, it recovers biologically meaningful axes of variation: broad lineage decisions are captured cleanly, while ambiguity is concentrated in populations that are biologically similar or transitional.

Taken together, these observations support the view that the supervised model is not merely overfitting technical artefacts but is sensitive to genuine biological structure; the

remaining errors mostly occur where the underlying biology itself does not provide sharp boundaries.

3.5 Summary and conclusions

In this chapter we developed and evaluated a supervised learning pipeline for automatic annotation of single-cell RNA-seq profiles into ten hematopoietic populations. The approach combines a rich but structured search over preprocessing choices and model families with an explicitly leak-free nested cross-validation strategy. By treating each pipeline configuration as a fixed “trial” and using inner CV as a stress-test rather than a hyperparameter tuner, we are able to obtain robust performance estimates despite exploring only a tiny fraction of the full combinatorial space.

The main quantitative findings can be summarised as follows. First, the choice of input representation is crucial: selecting around 1500-2000 HVGs and projecting them onto 20-30 principal components yields the best trade-off between information and noise. Gene filtering, cell-cycle regression and marker-score augmentation have secondary effects, with pre-filtering and medium filtering showing slight advantages but no universally dominant configuration. Second, tree-based ensembles and regularised linear models perform best across the nested-CV experiments, with the final ExtraTrees pipeline achieving a macro-F1 of about 0.44. Non-linear kernel SVMs and Gaussian Naive Bayes lag behind, while MLPs can match the leaders only for carefully chosen architectures.

Finally, the meta-analysis of hyperparameter importance confirms that data-first decisions, how many genes and components to include, and which representation to use, are more impactful than the fine details of any particular algorithm, at least within the limitations of our explored combinations. Together with the biological error analysis, these results show that relatively standard classifiers, when paired with thoughtful preprocessing and a rigorous evaluation protocol, can learn biologically meaningful decision rules for complex single-cell data.

Final considerations

In this project, we conducted a comprehensive evaluation of unsupervised and supervised machine-learning methods. Among the four dimensionality-reduction techniques explored, PCA, t-SNE, UMAP and autoencoders, the autoencoder-based representation proved to be the most effective, achieving the highest silhouette score of 0.603 and outperforming PCA’s more modest linear separation, the locally focused but globally distorted t-SNE embeddings, and even the strong UMAP configurations, which reached silhouettes up to 0.558. When paired with clustering algorithms, the best overall unsupervised pipeline combined the autoencoder latent space with agglomerative clustering using single

linkage and Euclidean distance, yielding the clearest and most compact cluster separation across all tested models.

In the supervised setting, a wide range of classifiers, logistic regression, SVMs, MLPs, Random Forests, ExtraTrees, XGBoost and GaussianNB, was assessed through rigorous nested cross-validation, and ExtraTrees consistently emerged as the top performer with a macro-F1 score of approximately 0.44. This model performed best when trained with a carefully optimised preprocessing strategy involving medium biological filtering applied before selecting around 1,750 highly variable genes, followed by PCA reduction to 20 principal components, and without additional marker scores or cell-cycle regression. The optimal ExtraTrees configuration used 400 trees with a maximum depth of 40 and a minimum split size of 5, striking a balance between expressive decision boundaries and robust generalisation. Together, these results demonstrate that combining biologically informed filtering, nonlinear dimensionality reduction, and tree-based supervised models yields strong and biologically meaningful performance for automated annotation of single-cell transcriptomic data.

A last consideration is that it is not surprising that the methods used to prepare the dataset before applying supervised techniques have such a significant impact on the exploration of the models' hyperparameters. This is in contrast to custom-built biomedical filters, which underperformed in almost all cases.

Finally, it would be interesting to try applying the same concepts to a larger population of cells, as well as to get access to some gpu clusters that could allow us to explore a wider range of possibilities.